Two-Server Sublinear PIR with Symmetric Privacy and Statistical Security

Shuaishuai Li[1], Anyu Wang[2(✉)], Cong Zhang[2], and Xiaoyun Wang[2]

[1] Zhongguancun Laboratory
liss@zgclab.edu.cn
[2] Tsinghua University
{zhangcong,anyuwang,xiaoyunwang}@tsinghua.edu.cn

Abstract. The field of private information retrieval (PIR) has made significant strides with a recent focus on protocols that offer sublinear online time, ensuring efficient access to public databases without compromising the privacy of the queries. The pioneering two-server PIR protocols developed by Corrigan-Gibbs and Kogan (EUROCRYPT 2020) enjoy the dual benefits of sublinear online time and statistical security. This allows their protocols to provide high efficiency and resist computationally unbounded adversaries. In this work, we extend this seminal work to the symmetric PIR (SPIR) context, where the protocol must ensure that the client is privy only to the requested database entries, with no knowledge of the remaining data. This enhancement aligns with scenarios where the confidentiality of non-requested information is as critical as the query itself. Our main result is the introduction of the first two-server SPIR protocols that achieve both sublinear online time and statistical security, together with an enhancement for achieving sublinear amortized time. Our protocols require a pragmatic level of shared randomness between the servers, which however is necessary for implementing statistical security in two-server SPIR, as showed by Gertner et al. (STOC 1998).

1 Introduction

Private information retrieval (PIR), introduced by Chor, Goldreich, Kushilevitz, and Sudan [12] in 1995, endows clients with the capability to access entries from a public database with confidentiality, ensuring that the specific location of the desired data entry remains secret. The domain of PIR typically encompasses two distinct scenarios: single-server and multi-server.

In the single-server setting, the client can access only a single server that owns the database. A straightforward approach to single-server PIR would be for the server to transmit the entire database to the client. Nonetheless, this method is impractical, particularly when the database is large, due to its linear communication cost. Consequently, researchers in the PIR domain aspire to develop protocols that achieve sublinear communication complexity [10–13,24]. It has been established that any non-trivial single-server PIR protocol, which offers sublinear communication, cannot be statistically secure [5, 17].

The multi-server PIR setting bypasses this limitation, with the prerequisite that the client can access multiple non-colluding servers holding replicas of the database. Multi-server PIR represents a weaker form of the single-server variant in the sense that should all servers collude, the scenario reverts to that of single-server PIR. However, the advantage of multi-server PIR is that we can achieve the dual benefits of sublinear communication and statistical security [12]. This enables the protocol to withstand adversaries with unbounded computational resources while still maintaining efficient communication.

PIR with sublinear online time. In the nascent stages of PIR development, the primary focus was on minimizing communication complexity [3,4,6,9,10,12,13,19–21,24,33,34,43]. However, as the field develops, it became increasingly apparent that the computational complexity of PIR protocols was equally critical to their real-world applicability. The work [7, 8] by Beimel, Ishai, and Malkin, is the first to address the linear computational complexity in PIR. Unfortunately, they encountered an inherent limitation: linear computational complexity is unavoidable for PIR protocols. Intuitively, if some database entry is not involved in the computation, the server knows that the client is not interested in this entry, which leaks information about the client's query index. To overcome the limitation of linear computation, they introduced the preprocessing model for PIR. This model permits

the protocol to undergo a query-independent offline stage before handling queries from the client. With an appropriately designed offline stage, they showed that each subsequent query could be answered in *sublinear* time. Specifically, they presented a two-server PIR protocol that offers statistical security and online complexity of $n^{0.5+\varepsilon}$ per query for any $\varepsilon > 0$. However, their protocol came with the tradeoff of substantial server-storage requirements. For example, to achieve $n^{0.6}$ online time, the servers need to store an encoding of size $N = n^{3.2}$. Even for modest database sizes (e.g., $n = 2^{20}$), the encoding would be much too large to materialize in practice.

Recently, Corrigan-Gibbs and Kogan [16] proposed novel two-server PIR protocols in the preprocessing model that enjoy sublinear online time without incurring the excessive server-storage costs, at the expense of modest client-storage — merely $\tilde{O}(n^{1/2})$. In the statistical setting, they first introduced a protocol that supports only a single query, which falls short of achieving sublinear *amortized* time. Then, they pointed out that by integrating the PIR-balancing technique of [12] with their protocol, one can obtain a statistical two-server PIR protocol with sublinear amortized time.

Symmetric privacy in PIR. PIR was initially devised to protect the privacy of clients when they access the database, ensuring that their queries do not reveal which data they are interested in. However, in many contexts where data sensitivity is paramount (e.g., the data market [1, 2]), the privacy of the database is equally crucial. For this reason, Gertner, Ishai, Kushilevitz, and Malkin [25] initialized the study of symmetric privacy in PIR. They introduced symmetric PIR (SPIR), which builds upon the foundation of PIR by imposing an additional layer of privacy: the client should only obtain the entries it has requested.

Like PIR, SPIR is also characterized by two variants: single-server and multi-server ones. Existing research [14, 25, 32] has shown that single-server SPIR cannot be statistically secure *even if linear or higher communication is allowed*. However, this is not the case for two-server SPIR. Gertner, Ishai, Kushilevitz, and Malkin [25] demonstrated that by leveraging shared (private) randomness between the servers, it is possible to construct two-server SPIR protocols that offer both *sublinear communication* and *statistical security*. In particular, they proved that shared randomness is unavoidable for achieving statistical security in two-server SPIR.

Our goal. In light of the developments in sublinear PIR (i.e., PIR with sublinear online time in the preprocessing model), we delve into the study of sublinear SPIR, with a focus on statistical security. It is worth noting that if statistical security is not a concern, we could employ the black-box conversion of [23] to transform two-server PIR protocols with sublinear online time (e.g., the protocols developed by Corrigan-Gibbs and Kogan [16]) into two-server SPIR protocols. This conversion would maintain the original protocols' asymptotic complexity. However, this approach relies on pseudorandom functions (PRFs), thus failing to achieve statistical security in the resulting SPIR protocol. In a nutshell, we pose the following question:

Can we construct two-server SPIR protocols with sublinear online time and statistical security? Moreover, can we further achieve sublinear amortized time?

1.1 Our Contributions

We answer the above question affirmatively by presenting statistical two-server SPIR protocols that achieve sublinear online time. Our protocols make an additional assumption that the two servers share some private randomness that is unknown to the client. This assumption has been proved to be unavoidable for achieving statistical security in two-server SPIR [25]. In practice, the shared randomness can be generated using interactions between the servers, i.e., one server directly samples the randomness and sends them to the other server. We note that the shared randomness inherently requires extra server-storage. However, unlike previous sublinear PIR works [7, 8] that suffer from very large extra server-stroage ($n^{3.2}$ for achieving $n^{0.6}$ online time), our protocols require only *sublinear* extra server-storage.

As our first result, we show how to equip the two-server PIR protocol of [16] with symmetric privacy while maintaining its statistical security and asymptotic complexity. This way, we obtain the first statistical two-server SPIR protocol with $\tilde{O}(n^{1/2})$ online time[3]. Concretely, we have the following theorem.

[3] Hereafter, we use $\tilde{O}(\cdot)$ to hide polylogarithmic factors in $O(\cdot)$.

Theorem 1 (Single-Query SPIR). *For a database with n entries, there exists a statistical two-server SPIR protocol supporting a single query in the preprocessing model, where*

- *The offline communication is $\tilde{O}(n^{1/2})$; the offline computation is $\tilde{O}(n)$.*
- *The online communication and computation are both $\tilde{O}(n^{1/2})$.*
- *The servers need to share and store $\tilde{O}(n^{1/2})$ randomness.*
- *The client-storage is $\tilde{O}(n^{1/2})$.*
- *The online query is completed in a single roundtrip (the client sends a single message to each server in parallel and receives an answer from each server).*

Our first SPIR protocol supports only a single query. We further strengthen it to support an unbounded number of queries. The resulting protocol requires the servers to share and store $\tilde{O}(n^{1/2}+\beta)$ randomness for answering β queries. To maintain $\tilde{O}(n^{1/2})$ extra server-storage, we recommend running the offline stage per $\Theta(n^{1/2})$ queries. Note that supporting $\Theta(n^{1/2})$ queries has allowed us to amortize the cost of the offline stage and achieve $\tilde{O}(n^{1/2})$ amortized time. We use the following theorem to show our result.

Theorem 2 (Multi-Query SPIR with Linear Client-Storage). *Let n be the database size, then there exists a statistical two-server SPIR protocol supporting β queries in the preprocessing model, where*

- *The offline communication is $\tilde{O}(n)$; the offline computation is $\tilde{O}(n)$.*
- *The online communication and computation per query are both $\tilde{O}(n^{1/2})$.*
- *The servers need to share and store $\tilde{O}(n^{1/2}+\beta)$ randomness.*
- *The client-storage is $\tilde{O}(n)$.*
- *Each online query is completed in a single roundtrip.*

Remark 1 (On the non-triviality of SPIR with linear communication). The multi-query scheme in Theorem 2 requires linear offline communication and client-side storage. Despite this, Theorem 2 is still a non-trivial result. Recall that PIR does not protect database privacy, hence a trivial PIR protocol would be for the client to directly download and save the entire database during the offline stage. However, since SPIR protects database privacy, it does not allow any database information leakage during the offline stage. Consequently, there is no such a trivial SPIR protocol.

Further, we demonstrate that sublinear offline communication and client-side storage are possible in two-server statistical SPIR with sublinear amortized time. Concretely, we propose a balancing technique for two-server SPIR and apply it to the above multi-query SPIR protocol. The resulting protocol requires the servers to share and store $\tilde{O}(n^{2/3}+\beta n^{1/3})$ randomness for answering β queries. We can let the parties run the offline stage per $\Theta(n^{1/3})$ queries, achieving $\tilde{O}(n^{2/3})$ extra server-storage. Supporting $\Theta(n^{1/3})$ queries allows us to amortize the cost of the offline stage and achieve $\tilde{O}(n^{2/3})$ amortized time.

Theorem 3 (Multi-Query SPIR with Sublinear Client-Storage). *For a database with size n, there exists a statistical two-server SPIR protocol supporting β queries in the preprocessing model, where*

- *The offline communication is $\tilde{O}(n^{2/3})$; the offline computation is $\tilde{O}(n)$.*
- *The online communication per query is $\tilde{O}(n^{1/3})$; the online computation per query is $\tilde{O}(n^{2/3})$.*
- *The servers need to share and store $\tilde{O}(n^{2/3}+\beta n^{1/3})$ randomness.*
- *The client-storage is $\tilde{O}(n^{2/3})$.*
- *Each online query is completed in a single roundtrip.*

Table 1 compares our protocols with the most relevant (S)PIR protocols with *statistical security*. We remark that the protocol of [25] works in the non-preprocessing model. For the work of [7, 8], we use the estimated data provided by the authors in their paper.

Remark 2 (Why starting from the seminal protocols of [16]). Since the foundational work of [16], there have been a bucket of optimizations [26, 31, 36, 40] in the two-server setting. However, these protocols rely on computationally secure primitives such as pseudorandom functions (PRFs) and therefore fail

Metrics		[25]	[7, 8]	[16]	[16]	Section 3	Section 4	Section 5
Offline	Comp.	0	$\mathsf{poly}(n)$	$\tilde{O}(n)$	$\tilde{O}(n)$	$\tilde{O}(n)$	$\tilde{O}(n)$	$\tilde{O}(n)$
	Comm.	0	$o(n)$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{2/3})$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n)$	$\tilde{O}(n^{2/3})$
Online	Comp.	$\tilde{O}(n)$	$n^{0.6+o(1)}$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{2/3})$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{2/3})$
	Comm.	$\tilde{O}(n^{1/3})$	$n^{0.6+o(1)}$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{1/3})$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{1/3})$
Server-storage		$\tilde{O}(n^{1/3})$	$n^{3.2+o(1)}$	0	0	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{2/3})$
Client-storage		0	0	$\tilde{O}(n^{1/2})$	$\tilde{O}(n^{2/3})$	$\tilde{O}(n^{1/2})$	$\tilde{O}(n)$	$\tilde{O}(n^{2/3})$
Multi-query		✓	✓	×	✓	×	✓	✓
Symmetric		✓	×	×	×	✓	✓	✓

Table 1. Comparison of two-server (S)PIR protocols with *statistical security*. For our first multi-query scheme (Section 4), the parties run the offline stage per $\Theta(n^{1/2})$ queries. For our second multi-query scheme (Section 5), the parties run the offline stage per $\Theta(n^{1/3})$ queries. For [25] and our schemes, "Server-storage" means the required storage space for storing the shared randomness. "Comp." is short for "Computation", and "Comm." is short for "Communication".

to achieve statistical security. The only exception is that Ishai, Shi, and Wichs [30] studied statistical single-server PIR and claimed that their results imply a statistical two-server PIR protocol with sublinear amortized time. However, their protocol achieves the same asymptotic complexity as [16] (i.e., $\tilde{O}(n^{2/3})$ amortized time). Therefore, the protocols of [16] are in fact the state-of-the-art two-server PIR with sublinear online time and *statistical security*.

Finally, we highlight that our work offers a new perspective on how sublinear online time is achieved. In sublinear PIR, a common intuition is that the sublinear online time is enabled by the client holding partial database information ($\tilde{O}(\sqrt{n})$ hints) in advance. This intuition appears to make sublinear SPIR impossible, as SPIR preprocessing strictly prohibits any leakage of database information to the client during the offline stage. However, our work reveals that sublinear online time is fundamentally achieved by the servers performing the inherent linear computation during the offline stage, rather than relying on the client's prior knowledge of the database.

To concretize this insight, we propose a modification to the PIR scheme in [16] that eliminates client-side database leakage during preprocessing while preserving sublinear online time. In the original scheme, each hint consists of a subset S and a hint value h, where h is the XOR of the database entries in S, computed by one of the servers. We modify the scheme such that the server computes but withholds the hint values during the offline stage. This way, the client learns nothing about the database, since the hint sets S alone reveal no information about the database. Yet, the protocol remains sublinear: the server can transmit the precomputed hint values during the online stage, which incurs only $\tilde{O}(n^{1/2})$ communication.

1.2 Revisiting the Statistical Two-Server PIR Protocols of [16]

In this section, we briefly revisit the statistically secure two-server PIR protocols of Corrigan-Gibbs and Kogan [16]. Their single-query protocol achieves $\tilde{O}(n^{1/2})$ online time and requires $\tilde{O}(n^{1/2})$ offline communication and client storage. We also describe a naive extension of their protocol to the multi-query setting. This protocol has linear offline communication and client storage. From now on, CL denotes the client. Moreover, LS and RS denote the two (left and right) servers.

Single-query PIR with $\tilde{O}(n^{1/2})$ online time. We first give an outline for the single-query PIR protocol of [16].[4]

Offline stage. The server takes a database $\mathsf{DB} = \{\mathsf{DB}_i\}_{i\in[\![n]\!]}$.

1. CL samples a random size-$n^{1/2}$ subset S of $[\![n]\!]$ and $w = (\ln 4)n^{1/2}$ random shifts[5] $\delta_1, \ldots, \delta_w \in [\![n]\!]$.

[4] Hereafter, we denote $[\![n]\!] = \{0, 1, \ldots, n-1\}$ and $[n] = \{1, \ldots, n\}$.

[5] The selection of w is to guarantee that the correct probability is no less than $1/2$. As a result, by running the protocol λ times, we can get a correct probability $1 - 1/2^{\lambda}$.

2. CL sends $S, \{\delta_j\}_{j\in[w]}$ to LS.
3. LS computes $S_j = \{i + \delta_j\}_{i\in S}$ and $h_j = \sum_{i\in S_j} \mathsf{DB}_i$ for all $j \in [w]$.
4. LS sends the hint values $\{h_j\}_{j\in[w]}$ to CL.
5. CL stores $S, \{(\delta_j, h_j)\}_{j\in[w]}$ as the hint table.

Online stage. For an index $\alpha \in [\![n]\!]$, the online stage proceeds as follows.

1. Query. The client generates the query messages as follows.
   (a) If $\alpha - \delta_j \notin S$ for all $j \in [w]$, then CL samples a random $s \in S$ and computes $\delta_{\mathsf{query}} = \alpha - s, S_{\mathsf{query}} = \{i + \delta_{\mathsf{query}}\}_{i\in S}$ and $k = 0$. Otherwise, CL finds the smallest $k \in [w]$ subject to $\alpha - \delta_k \in S$ and sets $S_{\mathsf{query}} = \{i + \delta_k\}_{i\in S}$.
   (b) CL samples a single bit b, which equals 0 with probability $(n^{1/2} - 1)/n$ and 1 with the remaining probability.
   (c) CL computes S^* by removing α^* from S_{query}, where α^* is sampled as follows.
   - If $b = 0$, α^* is a random element in the set $S_{\mathsf{query}}\backslash\{\alpha\}$.
   - If $b = 1$, $\alpha^* = \alpha$.

   (d) CL sends S^* to RS.
2. Answer. RS computes and sends $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ to CL.
3. Extract. If $b = 1$ and $k > 0$, CL computes and returns $x_\alpha = h_k - h^*$. Otherwise, CL returns $\perp$.

We briefly demonstrate the security of the above protocol. We first argue the correctness. If $b = 1$ and $k > 0$, then $x_\alpha = h_k - h^* = \sum_{i\in S_k} \mathsf{DB}_i - \sum_{i\in S^*} \mathsf{DB}_i$. Since $S^* = S_k\backslash\{\alpha\}$, we know that $x_\alpha = \mathsf{DB}_\alpha$, i.e., the client obtain the desired output. The query fails when $b = 0$ or $k = 0$. The choice of w guarantees that the query will succeed with a probability of at least $1/2$.[6] By running λ copies of the protocol, the probability that the query fails will be at most $2^{-\lambda}$.

Now we consider the privacy of the protocol. For the left server, it sees only $(S, \delta_1, \ldots, \delta_w)$ in the offline stage. Apparently, $(S, \delta_1, \ldots, \delta_w)$ is independent of α and therefore leaks nothing about α. For the right server, it sees only S^* in the online stage. With a bit of work, one can show that S^* is a random set that is independent of α, which implies that the privacy w.r.t. the right server holds.

Offline complexity. We give a complexity analysis for the offline stage. Let l be the entry length of the database. In the offline stage, the client samples S and $\delta_1, \ldots, \delta_w$ and sends them to the left server, which takes $O(n^{1/2}\log n + w\log n) = \tilde{O}(n^{1/2})$ computation and communication. Then the left server computes and sends $\{h_j\}_{j\in[w]}$ to the client, which takes $wl(n^{1/2} - 1) = \tilde{O}(ln)$ computation and $wl = \tilde{O}(ln^{1/2})$ communication. Hence the total computation and communication are $\tilde{O}(ln)$ and $\tilde{O}(ln^{1/2})$, respectively. Finally, storing $(S, \{(\delta_j, h_j)\}_{j\in[w]})$ requires $n^{1/2}\log n + w(\log n + l) = \tilde{O}(ln^{1/2})$ client-storage.

Online complexity. In the online stage, to deal with the query, the client first checks whether $\alpha - \delta_j \notin S$ for all $j \in [w]$ at step (a). A naive method checking whether $\alpha - \delta_j = s$ for all $j \in [w], s \in S$ will take $O(wn^{1/2}\log n) = \tilde{O}(n)$ computation. By utilizing hash functions, the work of [16] pointed out that step (a) can be accomplished using only $\tilde{O}(n^{1/2})$ computation[7]. After the check procedure, the client needs to compute S_{query}, which takes $O(n^{1/2}\log n) = \tilde{O}(n^{1/2})$ computation. At step (b), the client can sample b by sampling $\log(n/(n^{1/2} - 1)) = O(\log n)$ uniformly random bits. When all of these bits are 0, the client sets $b = 0$. Apparently, the probability that $b = 0$ is $(1/2)^{\log(n/(n^{1/2}-1))} = (n^{1/2} - 1)/n$. This way, step (b) takes $O(\log n)$ computation. At step (c), the client needs to compute α^*, which takes $O(\log n)$ computation. In total, the computation of generating the query messages is $\tilde{O}(n^{1/2})$. Finally, it is easy to see that sending S^* takes $\tilde{O}(n^{1/2})$ communication.

After receiving S^*, the right server computes and sends $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ to the client. This takes $\tilde{O}(ln^{1/2})$ computation and $O(l)$ communication. Moreover, the Extract procedure takes $\tilde{O}(1)$ computation.

Multi-query PIR with linear client storage. The multi-query protocol differs from the above single-query protocol mainly in the following aspects.

[6] See the proofs for our single-query schemes (Appendixes A and B) for more details.

[7] In fact, we find that the use of hash functions could be avoided, thus eliminating the additional storage requirement for storing the hash tables. Concretely, we let the client sort S in some lexicographic order during the offline stage, which takes only $\tilde{O}(n^{1/2})$ computation by utilizing a standard efficient sorting algorithm such as Quicksort [28]. To check whether $\alpha - \delta_j \in S$ for all $j \in [w]$, we can use the binary search algorithm, which takes only $O(w\log n) = \tilde{O}(n^{1/2})$ computation.

1. The hint sets are replaced with truly random sets. As a result, the offline communication and client-storage become linear.
2. A refresh procedure is used to request a new hint for each query.

The formal description is in the following.

Offline stage. The server takes a database $\mathsf{DB} = \{\mathsf{DB}_i\}_{i\in[\![n]\!]}$.

1. CL samples $w = (\ln 4)n^{1/2}$ random size-$n^{1/2}$ subsets $S_1, \ldots, S_w$ of $[\![n]\!]$.
2. CL sends $S_1, \ldots, S_w$ to LS.
3. LS computes $h_j = \sum_{i\in S_j} \mathsf{DB}_i$ for all $j \in [w]$.
4. LS sends the hint values $\{h_j\}_{j\in[w]}$ to CL.
5. CL stores the hint table $\{(S_j, h_j)\}_{j\in[w]}$.

Online stage. For each query $\alpha \in [\![n]\!]$, the online stage proceeds as follows.

1. Query. The client generates the query messages as follows.
   (a) CL samples a random size-$(n^{1/2}-1)$ set $S \subseteq [\![n]\!]\backslash\{\alpha\}$ and computes $S_{\mathsf{new}} = S \cup \{\alpha\}$.
   (b) If $\alpha \notin S_j$ for all $j \in [w]$, then CL computes $k = 0$. Otherwise, CL finds the smallest $k \in [w]$ subject to $\alpha \in S_k$.
   (c) CL samples a single bit b, which equals 0 with probability $(n^{1/2}-1)/n$ and 1 with the remaining probability.
   (d) If $b = 1$ and $k > 0$, CL sets $S_{\mathsf{query}} = S_k$. Otherwise, CL sets $S_{\mathsf{query}} = S_{\mathsf{new}}$.
   (e) CL computes S^h by removing α^* from S_{new}, and S^* by removing α^* from S_{query}, where α^* is sampled as follows.
      - If $b = 0$, α^* is a random element in the set $S_{\mathsf{query}}\backslash\{\alpha\}$.
      - If $b = 1$, $\alpha^* = \alpha$.
   (f) CL sends S^h to LS and S^* to RS.
2. Answer. LS computes and sends $h = \sum_{i\in S^h} \mathsf{DB}_i$ to CL; RS computes and sends $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ to CL.
3. Extract. If $b = 1$ and $k > 0$, CL computes and returns $x_\alpha = h_k - h^*$. Otherwise, CL returns $\perp$.
4. Refresh. If $b = 1$ and $k > 0$, CL replaces (S_k, h_k) with $(S_{\mathsf{new}}, x_\alpha + h)$.

We briefly demonstrate the correctness. For the first query, the discussion is similar to that for the single-query protocol. For the subsequent queries, with a bit of work, one can show that the hint distribution remains unchanged after each query. As a result, each subsequent query will succeed with the same probability as the first query. That is, each query will succeed with a probability of at least 1/2. By running λ copies of the protocol, the probability that the query fails will be at most $2^{-\lambda}$.

Now we consider the privacy of the protocol. For the left server, it sees $S_1, \ldots, S_w$ during the offline stage and S^h during the online stage. The values $S_1, \ldots, S_w$ are independent of α and leak nothing about α. With a bit of work, one can show that S^h is a random set of size $\sqrt{n} - 1$ and thus leaks nothing about α. Therefore, the privacy w.r.t. the left server holds. For the right server, it sees only S^* in the online stage. One can also show that S^* is a random set of size $\sqrt{n} - 1$. As a result, the privacy holds.

Using a similar discussion to that for the aforementioned single-query scheme, we can conclude the offline and online complexities as follows (let l be the entry length of the database).

- Offline complexity. The computation is $\tilde{O}(ln)$, and the communication is $\tilde{O}(ln^{1/2}+n)$. Additionally, the client-storage is $O(ln^{1/2} + n)$.
- Online complexity. The computation per query is $\tilde{O}(ln^{1/2})$, and the communication per query is $\tilde{O}(l + n^{1/2})$.

We remark that Corrigan-Gibbs and Kogan [16] pointed out that using the PIR-balancing technique of [12], one can reduce the offline communication and client-storage of their multi-query scheme to $\tilde{O}(n^{2/3})$ (at the price that the amortized time increases to $\tilde{O}(n^{2/3})$). However, this technique does not apply to SPIR. We will show how to obtain similar results in the SPIR context tailored to the two-server setting.

Remark 3 (Trade offline server-work for client-work). During the offline stage, we let the client sample the hint sets and send them to the left server. In fact, since the client has no inputs during the offline stage, the left server can directly sample the hint sets and compute the corresponding hint values. This way, the client only needs to store the hints without sending anything.

1.3 Result 1: Statistical Single-Query SPIR with Sublinear Communication and Storage

As our first result, we show how to equip the single-query PIR protocol of [16] (see Section 1.2 for its description) with symmetric privacy while maintaining its asymptotic efficiency and statistical security. As a result, we obtain the first statistical two-server SPIR protocol with sublinear online time.

Technical outline. To equip the single-query PIR protocol of [16] with symmetric privacy, we deal with the following two challenges.

Challenge 1: hints leak information about the database. By the privacy requirement of SPIR, the client should not obtain any information about the database at the end of the offline stage. However, during the offline stage of the protocol of [16], the client downloads many hints from the left server. Since each hint is the sum of some database entries, downloading hints violates the privacy of SPIR.

Idea 1: using masked hints. Our approach for dealing with the above leakage is to use *masked* hints. That is, the left server uses a random value to mask each hint and sends the masked hints to the client. Note that the masked hints are in fact random values, hence leaking nothing about the database. At this point, it seems intuitively that the client cannot use these masked hints to speed up the online stage, as it only obtains "useless" random values. However, we observe that the essential role of hints is to enable the server to perform the *inherent linear computation* in the offline stage, which is not hindered *even using masked hints*. The question now is how the online stage proceeds with masked hints. Recall that in the online stage, each query may fail with non-negligible probability. We first show how to enable the client to obtain the desired output when the query succeeds (we will handle fail queries in the next challenge). In the original protocol, when the query succeeds, the client finds (S_k, h_k) such that $\alpha \in S_k$ and asks the right server for the value $h^* = \sum_{i \in S_k \setminus \{\alpha\}} \mathsf{DB}_i$. Once obtained h^*, the client can recover DB_α by computing $h_k - h^*$. With our idea of using masked hints, each hint will be of form $h_k = r_k + \sum_{i \in S_k} \mathsf{DB}_i$, where r_k is a random mask sampled by the left server. To enable the client to obtain the retrieved value, we let the client ask the right server for the value $h^* = r_k + \sum_{i \in S_k \setminus \{\alpha\}} \mathsf{DB}_i$. Computing h^* requires the knowledge of r_k and $S_k \setminus \{\alpha\}$, where the latter is sent by the client. The remaining problem is how to enable the right server to obtain r_k. Our idea is to let the left server share the masks with the right server during the offline stage (this is the shared randomness in our SPIR protocol). Then, we let the client send k to the right server. Since the distribution of k is independent of the query index α, leaking k will not influence the privacy of the query. This way, the right server is able to compute h^*.

Challenge 2: handling the leakage when the query fails. In the single-query protocol of [16], the query fails when $b = 0$ or $k = 0$. Intuitively, we need to prevent the servers from knowing whether the query fails, otherwise they would know information about the query index α. However, we find that this is not true for the right server in the failure case that $k = 0$ (which implies that $\alpha - \delta_j \notin S$ for all $j \in [w]$). We observe that $S, \{\delta_j\}_{j \in [w]}$ are sampled independent of α and unknown to the right server. Therefore, even the right server knows that $\alpha - \delta_j \notin S$ for all $j \in [w]$, no information about α is revealed. We remark that this observation works only for a single query. When there are multiple queries, the right server may obtain information about $S, \{\delta_j\}_{j \in [w]}$ during the first query, which allows it to infer information about subsequent queries. Since we allow the right server to know whether the query fails, the client can directly send $k = 0$ to the right server for the failure case that $k = 0$. The failure case that remains to be considered is that $k > 0, b = 0$. In this case, the client will remove α^* from the set S_k for some $\alpha^* \neq \alpha$. As a result, the client will obtain DB_{α^*}. Since $\alpha^* \neq \alpha$, this leaks information about the database to the client, violating the database privacy of SPIR. We now show how to deal with this leakage.

Idea 2: masking the output. To prevent the client from knowing DB_{α^*} when $k > 0$ and $\alpha^* \neq \alpha$, we let the client obtain $\hat{r} \cdot (\alpha - \alpha^*) + \mathsf{DB}_{\alpha^*}$ instead of DB_{α^*}. The correctness still holds, as $\hat{r} \cdot (\alpha - \alpha^*) + \mathsf{DB}_{\alpha^*} = \mathsf{DB}_\alpha$ when $\alpha^* = \alpha$. If $\hat{r}, \alpha, \alpha^*, \mathsf{DB}_{\alpha^*}$ belong to a large field $\mathbb{F}$ (i.e., $\log |\mathbb{F}| \geq \lambda$), then by setting $\hat{r}$ to a random non-zero field element, we know that $\hat{r} \cdot (\alpha - \alpha^*) + \mathsf{DB}_{\alpha^*}$ is statistically indistinguishable from a random element when $\alpha^* \neq \alpha$, which guarantees privacy. The left problem is how to compute $\hat{r} \cdot (\alpha - \alpha^*) + \mathsf{DB}_{\alpha^*}$. We first let the servers share the random value $\hat{r}$ (i.e., $\hat{r}$ is a shared randomness). Then, we let the client secret-share $\alpha - \alpha^*$ additively among the two servers such that any single server knows nothing about $\alpha - \alpha^*$. Assume the sharing is (α_0, α_1), then the servers can compute an additive sharing of

$\hat{r} \cdot (\alpha - \alpha^*) = \hat{r} \cdot \alpha_0 + \hat{r} \cdot \alpha_1$. By letting the servers add a fresh zero-sharing $(\tilde{r}, -\tilde{r})$ to the final answer tuple, we can guarantee that the client obtains only $\hat{r} \cdot (\alpha - \alpha^*) + \mathsf{DB}_{\alpha^*}$ from the answers.

Remark 4 (Creating shared randomness without server-to-server interactions). In practice, the servers can generate the shared randomness between the servers by allowing them to communicate with each other. However, if computational security of the offline stage is acceptable (the online stage is still statistically secure), then we can eliminate server-to-server communication. A naive method is to use public-key encryption (PKE) by letting the client forward public keys and encrypted randomness. However, this requires a high round complexity: a server must obtain the public key of the other server before encrypting and sending the randomness. We can generate the shared randomness in a single roundtrip using a single-pass[8] key agreement protocol such as the Diffie-Hellman key exchange protocol [18], that is, the client transmits the messages between the servers.

1.4 Result 2: Statistical Multi-Query SPIR with Linear Communication and Storage

We have shown how to obtain a two-server SPIR protocol with sublinear online time and statistical security. The drawback of this protocol is that it supports only a single query, thus failing to achieve sublinear amortized time. Our second result is a statistical two-server SPIR protocol that supports multiple queries. The disadvantage of this protocol is that it requires linear offline communication and client storage. We will show how to optimize this in the next subsection.

Technical outline. We start from the statistical multi-query PIR protocol implied by [16] (see Section 1.2 for its description). Similar to the single-query setting, we use masked hints to prevent the leakage introduced by the hints. However, to support multiple queries, we still need to deal with some further challenges, as outlined below.

Challenge 1: revealing k hinders multi-query. As we have stated, revealing whether $k = 0$ may leak information about the queries when there are multiple queries. In fact, hiding only whether $k = 0$ is not sufficient for guaranteeing security. To illustrate this, we assume that the client makes two sequential queries α and α' and using the same k (which happens with non-negligible probability). Let S_1^* and S_2^* be the query sets sent to the right server for the two queries. If the right server knows that the two queries use the same $k > 0$, then the right server knows that the set $S_1^* \cup \{\alpha\}$ in the hint table is replaced with $S_2^* \cup \{\alpha'\}$, which implies that $\alpha \in S_2^* \cup \{\alpha'\}$. As a result, the right server knows that $\alpha \in S_2^*$ or the two queries are identical. This violates the query privacy.

Idea 1: protecting the privacy of k. To deal with the leakage introduced by revealing k in the multi-query setting, we manage to protect k from leaking to the right server. There are two cases that we need to consider: $k = 0$ and $k > 0$. We first consider the case that $k > 0$. Recall that the role of k is to enable the right server to compute r_k such that the mask in h_k can be offset. Assume that $\boldsymbol{r}$ is the mask vector held by the two servers. Let $(a_0, a_1 + r_k)$ be the final answer tuple from the servers ($a_0 + a_1 + r_k$ is the required value for recovering the output). Instead of sending k to the right server, we let the parties run a secure three-party computation protocol to compute $a_1 + r_k$, where $\boldsymbol{r}$ and k are the private inputs. The idea is to let the client generate an additive sharing $(\boldsymbol{v}_0, \boldsymbol{v}_1)$ of the k-th unit vector $\boldsymbol{e}_k$ (where the k-th entry is 1 and other entries are 0). Here, $\boldsymbol{v}_0$ (resp. $\boldsymbol{v}_1$) will be part of the query message sent to the left (resp. right) server. This way, the servers can compute an additive sharing of $r_k = \langle \boldsymbol{r}, \boldsymbol{v}_0 \rangle + \langle \boldsymbol{r}, \boldsymbol{v}_1 \rangle$ (note that both servers know $\boldsymbol{r}$). The final answer tuple is $(a_0 + \langle \boldsymbol{r}, \boldsymbol{v}_0 \rangle, a_1 + \langle \boldsymbol{r}, \boldsymbol{v}_1 \rangle)$. Note that $a_0 + \langle \boldsymbol{r}, \boldsymbol{v}_0 \rangle + a_1 + \langle \boldsymbol{r}, \boldsymbol{v}_1 \rangle = a_0 + a_1 + r_k$, hence the client can recover the retrieved entry.

We now consider the case that $k = 0$. In this case, the query fails and we need to guarantee that the client obtains nothing. To achieve this, we let the servers maintain a dummy database entry $\bar{r}$. Recall that the answers from the servers are of form $(a_0 + \langle \boldsymbol{r}, \boldsymbol{v}_0 \rangle, a_1 + \langle \boldsymbol{r}, \boldsymbol{v}_1 \rangle)$, where $\boldsymbol{r}$ is the mask vector and $(\boldsymbol{v}_0, \boldsymbol{v}_1)$ is an additive sharing of $\boldsymbol{e}_k$. We let the servers replace $\boldsymbol{r}$ with $(\bar{r}, \boldsymbol{r})$. Moreover, the client replaces $(\boldsymbol{v}_0, \boldsymbol{v}_1)$ with an additive sharing of $(1, 0^{|\boldsymbol{r}|})$ if the query fails and $(0, \boldsymbol{e}_k)$ otherwise (note that the client knows whether the query succeeds before sending the query messages to the servers). In both cases, each server sees only a random vector. As a result, when the query succeeds, the client still obtains $a_0 + a_1 + \langle (\bar{r}, \boldsymbol{r}), (0, \boldsymbol{e}_k) \rangle = a_0 + a_1 + \langle \boldsymbol{r}, \boldsymbol{e}_k \rangle$. If the query fails, the client will

[8] A single-pass protocol requires that each party in the protocol sends only a single message simultaneously.

obtain $a_0 + a_1 + \langle(\bar{r}, \boldsymbol{r}), (1, 0^{|\boldsymbol{r}|})\rangle = a_0 + a_1 + \bar{r}$. By setting $\bar{r}$ to a random value, the client obtains nothing (that is, $\bar{r}$ is a shared randomness).

Challenge 2: the server cannot update the masks. To support multiple queries, the two servers need to update the mask vector $\boldsymbol{r}$ properly. When a successful query is completed, the client discards the consumed hint (S_k, h_k) and replace it with a new hint obtained from the left server. To ensure the correctness of the subsequent queries, the server must update the mask vector by replacing the corresponding mask r_k in $\boldsymbol{r}$ with a new mask corresponding to the new hint. When the query fails and no hint is consumed, the server should not update the mask vector. Apparently, updating the mask vector requires the knowledge of k. However, as we have discussed before, revealing k to the servers may violate the privacy when handling multiple queries. This brings us into a dilemma: protecting k prevents the server from correctly updating the mask vector (this implies that the servers cannot correctly answer subsequent queries), and revealing k violates the privacy required by SPIR.

Idea 2: appending new hints instead of replacing consumed hints. Our novel idea for dealing with this challenge is to let the server *append the new mask to the mask vector* instead of replacing the consumed mask from the mask vector. That is, let $\boldsymbol{r}$ be the original mask vector and r be the new mask that is used for the new hint, then the servers simply take $(\boldsymbol{r}, r)$ as the new mask vector. When the query succeeds and (S_k, h_k) is the consumed hint, the client just appends the new hint (S, h) to the original hint vector, rather than replacing (S_k, h_k) with (S, h). This way, the index of (S, h) is set to $|\boldsymbol{r}| + 1$ instead of k. Note that the client cannot use (S_k, h_k) in subsequent queries. To ensure that the client always take the unused hints, we let the client maintain a set M that contains the indexes of the unused hints. Initially, M contains the indexes of all the hints. Whenever a hint (S_k, h_k) is consumed and a new hint (S_m, h_m) is generated, the client updates M to $M \cup \{m\} \backslash \{k\}$. For each online query, we let the client always take a hint with index in M. Apparently, our new approach does not require the servers to know the exact value of k.

The main drawback of our approach is that the length of the mask vector increases linearly with the number of queries, which increases the server-storage requirement[9]. However, as long as the total number of queries is $\Theta(n^{1/2})$, we can guarantee that the extra server-storage requirement remains $\tilde{O}(n^{1/2})$. We remark that the ability to handle $\Theta(n^{1/2})$ queries already allows our protocol to achieve $\tilde{O}(n^{1/2})$ amortized time.

Remark 5 (A new single-query SPIR protocol without relying on large fields). Recall that in our single-query protocol, we require that the database entries belong to a large field, resulting in an λ overhead for small entries. The main reason for relying on large fields is that we need to mask the output, ensuring that the client obtains $\hat{r}(\alpha - \alpha^*) + \mathsf{DB}_{\alpha^*}$ instead of DB_{α^*}. If we adopt the same idea of hiding k as in our multi-query protocol (but the hint sets are still sampled in the same way as in the single-query scheme), then we no longer require to mask the output. The reason is that whenever the query fails, the client always obtains a random value $\bar{r}$. This gives us a single-query SPIR protocol that does not rely on large fields. See Section 3.2 for more details.

1.5 Result 3: A Balancing Technique for Two-Server SPIR

The drawback of our multi-query scheme is that it requires linear offline communication and client-storage. To deal with this, we utilize a balancing technique for two-server SPIR, which allows us to reduce the offline communication and client-storage to $\tilde{O}(n^{2/3})$.

Technical outline. Our technique is inspired by basic idea of [12], which accounts to split the database into multiple subdatabases. The key point is that the client can submit the same query index for each subdatabase. By choosing suitable parameters, we can reduce the client's complexity. However, we note that this technique does not apply seamlessly to SPIR because the client will obtain an entry from each subdatabase, while only one of these entries is its desired output. This violates the database privacy in SPIR. To deal with this problem, we show a way for the servers to return only the answer vector that the client wants. Specifically, assume that $\boldsymbol{a}_0$ and $\boldsymbol{a}_1$ are the answer vectors of the two servers, respectively. That is, the t-th entries of $\boldsymbol{a}_0$ and $\boldsymbol{a}_1$ are the answers for the t-th

[9] The increase in client-storage is minimal, as the consumed hints could be discarded. The additional client-storage is introduced by storing the indexes of the new hints. These indexes, however, is of size only logarithmic in $w + \beta$ for β queries.

subdatabase. If the client wants the j-th entry, then the idea is to have the client take $\boldsymbol{e}_j$ as input, and the parties securely compute the circuit $(\langle \boldsymbol{a}_0, \boldsymbol{e}_j \rangle, \langle \boldsymbol{a}_1, \boldsymbol{e}_j \rangle)$.[10] We will show how to achieve this in a single roundtrip by utilizing shared randomness between the servers. The following theorem states our result.

Theorem 4 (Balancing Two-Server SPIR). *Assume that there exists a two-server SPIR protocol with database size n such that*

- *The randomness size is $S_r(n)$.*
- *The query size is $S_q(n)$; the query time is $T_q(n)$.*
- *The answer size is $S_a(n)$; the answer time is $T_a(n)$.*
- *The extract time is $T_e(n)$.*
- *The protocol is completed in a single roundtrip.*

Then for any integer $n_r \in [n]$,[11] there exists a two-server SPIR protocol with database size n such that

- *The randomness size is $O(n_r) \cdot S_r(n/n_r)$.*
- *The query size is $S_q(n/n_r) + O(n_r)$; the query time is $T_q(n/n_r) + O(n_r)$.*
- *The answer size is $O(n_r) \cdot S_a(n/n_r)$; the answer time is $O(n_r) \cdot T_a(n/n_r)$.*
- *The extract time is $T_e(n/n_r) + O(n_r)$.*
- *The protocol is completed in a single roundtrip.*

In particular, if the original protocol is statistically secure, then the resulting protocol is still statistically secure.

With some effort, by setting $n_r = \Theta(n^{1/3})$, we can apply the above SPIR-balancing technique to our multi-query SPIR scheme and reduce the client-side offline complexity (computation, communication, storage) to $\tilde{O}(n^{2/3})$, at the price that the amortized time increases from $\tilde{O}(n^{1/2})$ to $\tilde{O}(n^{2/3})$. See Section 5 for more details.

1.6 Related Work

Since the pioneering work of [16], a series of optimizations have been proposed. We briefly revisit the related works within the PIR framework of [16], including both the single-server and multi-server settings. We remark that all of these works except [30, 41] rely on computationally secure primitives, thus failing to achieving statistical security.

PIR with client-preprocessing. Shi, Aqeel, Chandrasekaran, and Maggs [40] showed how to reduce the online communication to $\tilde{O}(1)$ under the learning with errors (LWE) assumption. At the same time, Kogan and Corrigan-Gibbs [31] showed how to achieve better concrete efficiency (i.e., lower dependence on the security parameter), but this scheme requires linear online computation or client-storage. Then, Corrigan-Gibbs, Henzinger, and Kogan [15] showed how to achieve sublinear amortized time in the single-server setting (the work of [16] only achieves sublinear amortized time in the two-server setting). Lazzaretti and Papamathou [36] showed how to optimize the polylogarithmic factors in the two-server setting and achieve better efficiency than [40] based on the DDH assumption. Zhou, Lin, Tselekounis, and Shi [44] and Lazzaretti and Papamathou [35] showed how to achieve $\tilde{O}(1)$ online communication in the single-server setting. Zhou, Park, Shi, and Zheng [38] and Ren, Mughees, and Sun [39] showed how to achieve better concrete efficiency without relying on public-key primitives. Ghoshal, Zhou, and Shi [26] further optimize theirs protocols and obtains reduced online communication using only one-way functions (OWFs). Lazzaretti and Papamathou [37] showed how to proceed the offline stage by scanning the database only once, thus obtaining better offline efficiency, but this scheme suffers linear client-storage. Ghoshal, Zhou, Shi, and Peng [27] showed how to achieve $\tilde{O}(1)$ online communication using only OWFs, by combining client-side and server-side preprocessing. Hoover, Patel, Persiano,

[10] It seems that a similar idea has been used in our first multi-query SPIR protocol. We argue that the situation is different. In our first multi-query SPIR protocol, the goal is to compute $r_k = \langle \boldsymbol{r}, \boldsymbol{e}_k \rangle$, where *both of the two servers know* $\boldsymbol{r}$. Nevertheless, $\boldsymbol{a}_0$ is known to *only one server* (and so is $\boldsymbol{a}_1$). Therefore, computing $\langle \boldsymbol{a}_0, \boldsymbol{e}_j \rangle$ in a single roundtrip will be more challenging.

[11] Hereafter, we assume that n_r divides n, otherwise we pad the database so that n is a multiple of n_r.

and Yeo [29] showed the first single-server PIR with client preprocessing that obtains optimal trade-offs between client storage and online time for all parameters. Previously, online time is bounded by $\tilde{\Omega}(n^{1/2})$. Fisch, Lazzaretti, Liu, and Papamanthou [22] showed how to achieve both sublinear offline communication and a low-depth preprocessing circuit. Finally, two works [30, 41] studied PIR with information-theoretic security. Singh, Wei, and Zikas [41] gave $2t$-server PIR protocols with statistical security for $t \geq 2$ that achieve $\tilde{O}(n^{1/2})$ online time. Ishai, Shi, and Wichs [30] explores the landscape of statistically secure single-server preprocessing PIR. Their results show that although classic results implies that single-server PIR with sublinear communication and statistical security is impossible [5, 17], one can bypass this lower bound by utilizing a query-independent offline stage with linear communication.

2 Preliminaries

Notations. Let λ be the statistical security parameter. For any integer n, we use $[n]$ and $[\![n]\!]$ to denote the sets $\{1, \cdots, n\}$ and $\{0, 1, \cdots, n-1\}$, respectively. We use bold letters to represent vectors. The k-th length-n unit vector $\boldsymbol{e}_k$ is a vector where the k-th entry is 1 and other entries are 0.

For two distributions X and Y over a finite domain $\mathcal{D}$, their statistical distance is defined as

$$\Delta(X, Y) = \frac{1}{2} \sum_{z \in \mathcal{D}} |\Pr[X = z] - \Pr[Y = z]|.$$

We say that two distributions are statistically indistinguishable if their statistical distance is negligible in λ.

2.1 Two-Server Symmetric Private Information Retrieval

In this section, we present the formal definition of two-server symmetric private information retrieval (SPIR). The definition is described in the multi-query and statistical setting.

Definition 1 (Two-Server SPIR). *Two-server SPIR is a three-party computation protocol proceeding among two servers (say, left and right servers) and a client. At the beginning, both of the two servers have a database $\mathsf{DB} = (\mathsf{DB}_1, \ldots, \mathsf{DB}_n)$, and the client has a query set $Q \subseteq [n]$. For each query $\alpha \in Q$, the client outputs a value x_α (the client may submit these queries one by one). For correctness consideration, we require that*

- *Correctness. For each $\alpha \in Q$, the probability that $x_\alpha \neq \mathsf{DB}_\alpha$ is negligible.*

For privacy consideration, we require that

- *Client privacy. The client obtains only the retrieved values $\{\mathsf{DB}_\alpha\}_{\alpha \in Q}$. Formally, there exists a simulator $\mathcal{S}$ such that the distribution $\mathcal{S}(Q, \{\mathsf{DB}_\alpha\}_{\alpha \in Q})$ is statistically indistinguishable from the client's view.*
- *Left-server privacy. The left server knows nothing about Q. Formally, there exists a simulator $\mathcal{S}$ such that the distribution $\mathcal{S}(\mathsf{DB})$ is statistically indistinguishable from the left server's view.*
- *Right-server privacy. The right server knows nothing about Q. Formally, there exists a simulator $\mathcal{S}$ such that the distribution $\mathcal{S}(\mathsf{DB})$ is statistically indistinguishable from the right server's view.*

The view of a party is defined as its input, random tape, and received messages.

Party notation. In the descriptions of our protocols, we always use CL to denote the client, and moreover, LS and RS denote the left and right servers.

3 Statistical Two-Server SPIR with Sublinear Online Time

We present two single-query SPIR protocols, where the first is based on the discussion in Section 1.3. The second protocol is obtained by replacing the strategy for handling failed queries in the first protocol with that used in our multi-query scheme (Section 1.4).

Compared to the first scheme, the advantage of the second scheme is that it does not rely on large fields. This makes it enjoy a better concrete efficiency when the entries are small. However, when the entry space is already a large field, the second scheme will have a higher (concrete) communication cost, as the client must send an additive sharing of a length-$O(\sqrt{n})$ vector in addition to the query set.

3.1 First Construction

In our first construction, we use the following public parameters.

- Let $\mathsf{DB} = \{\mathsf{DB}_i\}_{i\in[\![n]\!]}$ be the database. Let $\mathbb{F}$ be a large field and $\mathbb{F}^* = \mathbb{F}\backslash\{0\}$, where $\log|\mathbb{F}| = \max\{\lambda, |\mathsf{DB}_i|\}$. We view all the entries as field elements in $\mathbb{F}$.
- We treat $[\![n]\!]$ as the ring $\mathbb{Z}_n$ and assume $[\![n]\!] \subseteq \mathbb{F}$. For each $i \in [\![n]\!]$, we denote $\mathsf{lift}(i)$ the field element $i \in \mathbb{F}$.

Protocol 3.1: First Single-Query SPIR

Shared randomness. The servers share the following randomness.

- $w+1$ random values $r_1, \ldots, r_w, \tilde{r} \in \mathbb{F}$ where $w = (\ln 4)n^{1/2}$.
- A random value $\hat{r} \in \mathbb{F}^*$.

Offline stage

1. CL samples a random size-$n^{1/2}$ subset S of $[\![n]\!]$.
2. CL samples w random shifts $\delta_1, \ldots, \delta_w \in [\![n]\!]$.
3. CL sends $S, \{\delta_j\}_{j\in[w]}$ to LS.
4. LS computes $S_j = \{i + \delta_j\}_{i\in S}$ and $h_j = r_j + \sum_{i\in S_j} \mathsf{DB}_i$ for all $j \in [w]$.
5. LS sends $\{h_j\}_{j\in[w]}$ to CL.
6. CL stores $S, \{(\delta_j, h_j)\}_{j\in[w]}$.

Online stage

1. Query. For the index $\alpha \in [n]$, CL performs the following procedures.
   (a) CL samples a single bit b, which equals 0 with probability $(n^{1/2}-1)/n$ and 1 with the remaining probability.
   (b) If $\alpha - \delta_j \notin S$ for all $j \in [w]$, then CL sets $k = 0$. Otherwise, CL finds the smallest $k \in [w]$ subject to $\alpha - \delta_k \in S$ and compute $S_{\mathsf{query}} = \{i + \delta_k\}_{i\in S}$.
   (c) If $k = 0$, then CL samples a random $s \in S$ and computes $\delta_{\mathsf{query}} = \alpha - s$, $S_{\mathsf{query}} = \{i + \delta_{\mathsf{query}}\}_{i\in S}$.
   (d) CL computes $S^* = S_{\mathsf{query}}\backslash\{\alpha^*\}$, where α^* is sampled as follows.
      – If $b = 0$, α^* is a random element in the set $S_{\mathsf{query}}\backslash\{\alpha\}$.
      – If $b = 1$, $\alpha^* = \alpha$.
   (e) CL chooses random $\alpha_0 \in \mathbb{F}$ and computes $\alpha_1 = \mathsf{lift}(\alpha - \alpha^*) - \alpha_0$.
   (f) CL sends α_0 to LS.
   (g) If $k = 0$, CL sends $\perp$ to RS. Otherwise, CL sends (S^*, k, α_1) to RS.
2. Answer.
   (a) LS computes $a_0 = \hat{r} \cdot \alpha_0 + \tilde{r}$ and sends a_0 to CL.
   (b) If receiving (S^*, k, α_1), RS computes $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ and $a_1 = \hat{r} \cdot \alpha_1 - r_k - h^* - \tilde{r}$. RS sends a_1 to CL. Otherwise, RS sends a random value a_1 to CL.
3. Extract.
   (a) If $k > 0$ and $b = 1$, then CL computes and returns $x_\alpha = a_0 + a_1 + h_k$. Otherwise, CL returns $\perp$.

Theorem 5. *Protocol 3.1 is a statistically secure two-server SPIR protocol.*

Proof. Due to space limitation, the proof is deferred to Appendix A.

3.2 Second Construction

We now present our second single-query SPIR scheme. In contrast to the first scheme, we no longer rely on a large field. Concretely, we use the following public parameters.

- Let $\mathsf{DB} = \{\mathsf{DB}_i\}_{i\in[\![n]\!]}$ be the database with each $\mathsf{DB}_i \in \mathcal{D} = \{0,1\}^l$.
- We treat $[\![n]\!]$ as the ring $\mathbb{Z}_n$.

Protocol 3.2: Second Single-Query SPIR

Shared randomness. The servers share the following randomness.

- $w+2$ random values $r_1, \ldots, r_w, \tilde{r}, \bar{r} \in \mathcal{D}$ where $w = (\ln 4)n^{1/2}$. Denote $\boldsymbol{r} = (\bar{r}, r_1, \ldots, r_w)$.

Offline stage

1. CL samples a random size-$n^{1/2}$ subset S of $[\![n]\!]$.
2. CL samples w random shifts $\delta_1, \ldots, \delta_w \in [\![n]\!]$.
3. CL sends $S, \{\delta_j\}_{j\in[w]}$ to LS.
4. LS computes $S_j = \{i+\delta_j\}_{i\in S}$ and $h_j = r_j \oplus \sum_{i\in S_j} \mathsf{DB}_i$ for all $j \in [w]$.
5. LS sends $\{h_j\}_{j\in[w]}$ to CL.
6. CL stores $S, \{(\delta_j, h_j)\}_{j\in[w]}$.

Online stage

1. Query. For the index $\alpha \in [n]$, CL performs the following procedures.
   (a) CL samples a single bit b, which equals 0 with probability $(n^{1/2}-1)/n$ and 1 with the remaining probability.
   (b) If $b = 0$ or $\alpha - \delta_j \notin S$ for all $j \in [w]$, then CL samples a random $s \in S$ and computes $\delta_{\mathsf{query}} = \alpha - s, S_{\mathsf{query}} = \{i+\delta_{\mathsf{query}}\}_{i\in S}$ and $k = 0$. Otherwise, CL finds the smallest $k \in [w]$ such that $\alpha - \delta_k \in S$ and computes $S_{\mathsf{query}} = \{i+\delta_k\}_{i\in S}$.
   (c) CL computes $S^* = S_{\mathsf{query}} \backslash \{\alpha^*\}$, where α^* is sampled as follows.
   – If $b = 0$, α^* is a random element in the set $S_{\mathsf{query}} \backslash \{\alpha\}$.
   – If $b = 1$, $\alpha^* = \alpha$.
   (d) CL chooses random $\boldsymbol{v}_0 \in \mathbb{F}_2^{w+1}$ and computes $\boldsymbol{v}_1 = \boldsymbol{e}_{k+1}^{w+1} \oplus \boldsymbol{v}_0$, where $\boldsymbol{e}_{k+1}^{w+1}$ is the $(k+1)$-th length-$(w+1)$ unit vector.
   (e) CL sends $\boldsymbol{v}_0$ to LS and $(S^*, \boldsymbol{v}_1)$ to RS.
2. Answer.
   (a) LS computes $a_0 = \langle \boldsymbol{v}_0, \boldsymbol{r} \rangle \oplus \tilde{r}$.
   (b) LS sends a_0 to CL.
   (c) RS computes $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ and $a_1 = \langle \boldsymbol{v}_1, \boldsymbol{r} \rangle \oplus h^* \oplus \tilde{r}$.
   (d) RS sends a_1 to CL.
3. Extract.
   (a) If $k > 0$, then CL computes and returns $x_\alpha = a_0 \oplus a_1 \oplus h_k$. Otherwise, CL returns $\perp$.

Theorem 6. *Protocol 3.2 is a statistically secure two-server SPIR protocol.*

Proof. The proof is deferred to Appendix B.

Remark 6. We remark that in our second single-query protocol, for simplifying the description, $k = 0$ represents that the query fails, not just that $\alpha - \delta_j \notin S$ for all $j \in [w]$.

3.3 Complexity Analysis

We proceed to summarize the complexity of our two single-query protocols. Due to space limits, we defer the detailed analysis to Appendixes D.1 and D.2. For the first scheme, let $l = \log |\mathbb{F}|$, then its complexity is as follows.

- Randomness complexity. The size of the shared randomness is $O(ln^{1/2})$.
- Offline complexity. The total computation and communication are $\tilde{O}(ln)$ and $\tilde{O}(ln^{1/2})$, respectively. Moreover, the client-storage is $\tilde{O}(ln^{1/2})$.
- Online complexity. The total computation and communication are $\tilde{O}(ln^{1/2})$ and $\tilde{O}(l + n^{1/2})$, respectively.

For the second scheme, let l be the entry length, then its complexity is as follows.

- Randomness complexity. The size of the shared randomness is $O(ln^{1/2})$.
- Offline complexity. The total computation and communication are $\tilde{O}(ln)$ and $\tilde{O}(ln^{1/2})$, respectively. Moreover, the client-storage is $\tilde{O}(ln^{1/2})$.
- Online complexity. The total computation and communication are $\tilde{O}(ln^{1/2})$ and $\tilde{O}(l + n^{1/2})$, respectively.

4 Statistical Two-Server SPIR with Linear Client-Storage and Sublinear Amortized Time

In this section, we present our multi-query SPIR scheme. The description uses the following public parameters.

- Let $\mathsf{DB} = \{\mathsf{DB}_i\}_{i\in[\![n]\!]}$ be the database with each $\mathsf{DB}_i \in \mathcal{D} = \{0,1\}^l$.
- We treat $[\![n]\!]$ as the ring $\mathbb{Z}_n$.
- Assume the client has β queries.

Protocol 4: Multi-Query SPIR with Linear Client-Storage

Shared randomness. The servers share the following randomness.

- A random vector $\boldsymbol{r} = (r_1, \ldots, r_w) \in \mathcal{D}^w$ with $w = (\ln 4)n^{1/2}$.
- 3β random values $\{r_{w+u}, \bar{r}_u, \tilde{r}_u\}_{u\in[\beta]} \in \mathcal{D}$. Each tuple $(r_{w+u}, \bar{r}_u, \tilde{r}_u)$ is used for the u-th query.

Offline stage

1. CL samples w random size-$n^{1/2}$ subsets $S_1, \ldots, S_w$ of $[\![n]\!]$.
2. CL sends $S_1, \ldots, S_w$ to LS.
3. LS computes $h_j = r_j \oplus \sum_{i\in S_j} \mathsf{DB}_i$ for all $j \in [w]$.
4. LS sends $\{h_j\}_{j\in[w]}$ to CL.
5. CL stores $\{(S_j, h_j)\}_{j\in[w]}$.
6. CL initializes the hint index-set $M = [w]$.

Online stage

1. Query. For the u-th query $\alpha \in [\![n]\!]$, CL performs the following procedures.
   (a) CL samples a single bit b, which equals 0 with probability $(n^{1/2}-1)/n$ and 1 with the remaining probability.
   (b) CL samples a random size-$(n^{1/2}-1)$ subset S of $[\![n]\!]\backslash\{\alpha\}$ and sets $S_{\mathsf{new}} = S \cup \{\alpha\}$.
   (c) If $b = 0$ or $\alpha \notin S_j$ for all $j \in M$, then CL computes $S_{\mathsf{query}} = S_{\mathsf{new}}$ and $k = 0$. Otherwise, CL finds the smallest $k \in M$ such that $\alpha \in S_k$ and sets $S_{\mathsf{query}} = S_k$.
   (d) CL computes $S^h = S_{\mathsf{new}}\backslash\{\alpha^*\}$ and $S^* = S_{\mathsf{query}}\backslash\{\alpha^*\}$, where α^* is sampled as follows.
      – If $b = 0$, α^* is a random element in the set $S_{\mathsf{new}}\backslash\{\alpha\}$.
      – If $b = 1$, $\alpha^* = \alpha$.
   (e) CL chooses random $\boldsymbol{v}_0 \in \mathbb{F}_2^{w+u}$ and computes $\boldsymbol{v}_1 = \boldsymbol{e}_{k+1}^{w+u} \oplus \boldsymbol{v}_0$, where $\boldsymbol{e}_{k+1}^{w+u}$ is the $(k+1)$-th length-$(w+u)$ unit vector.
   (f) CL sends $(S^h, \boldsymbol{v}_0)$ to LS and $(S^*, \boldsymbol{v}_1)$ to RS.
2. Answer. Let $(r, \bar{r}, \tilde{r}) = (r_{w+u}, \bar{r}_u, \tilde{r}_u)$ and $\bar{\boldsymbol{r}} = (\bar{r}, \boldsymbol{r})$.
   (a) LS computes $h = r \oplus \sum_{i\in S^h} \mathsf{DB}_i$ and $a_0 = \langle \boldsymbol{v}_0, \bar{\boldsymbol{r}}\rangle \oplus \tilde{r}$.
   (b) LS sends (h, a_0) to CL.
   (c) RS computes $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ and $a_1 = \langle \boldsymbol{v}_1, \bar{\boldsymbol{r}}\rangle \oplus h^* \oplus \tilde{r}$.
   (d) RS sends a_1 to CL.
   (e) LS and RS update $\boldsymbol{r} \leftarrow (\boldsymbol{r}, r)$.
3. Extract.
   (a) If $k > 0$, then CL computes and returns $x_\alpha = a_0 \oplus a_1 \oplus h_k$. Otherwise, CL returns $\perp$.
4. Refresh.
   (a) If $k > 0$, then CL updates $M \leftarrow M \cup \{w+u\}\backslash\{k\}$ and stores $S_{w+u} = S_{\mathsf{new}}, h_{w+u} = x_\alpha \oplus h$.

Theorem 7. *Protocol 4 is a statistically secure two-server SPIR protocol.*

Proof. The proof is deferred to Appendix C.

4.1 Complexity Analysis

We summarize the complexity of the above multi-query protocol. The detailed analysis can be found in Appendix D.3. Let l be the entry length, for answering β queries, the complexity is as follows.

- Randomness complexity. The size of the shared randomness is $O(ln^{1/2} + l\beta)$.
- Offline complexity. The total computation and communication are $\tilde{O}(ln)$ and $\tilde{O}(n + ln^{1/2})$, respectively. Moreover, the client-storage is $\tilde{O}(n + ln^{1/2})$.
- Online complexity. For the u-th query, the total computation and communication are $\tilde{O}(ln^{1/2} + lu)$ and $\tilde{O}(l + n^{1/2} + u)$, respectively.

By letting the parties run the offline stage per $\Theta(n^{1/2})$ queries (i.e., $\beta = \Theta(n^{1/2})$), we can achieve $\tilde{O}(ln^{1/2})$ amortized time. The disadvantage of this protocol is that it has linear offline communication and client-storage. In the next section, we show how to achieve sublinear offline communication and client-storage.

5 A Balancing Technique for Two-Server SPIR

In this section, we introduce a balanced technique for two-server SPIR. As we will see, this technique allows us to obtain a multi-query SPIR with sublinear amortized time and client-storage.

5.1 Balancing Standard SPIR

As the starting point, we introduce a balancing technique for single-round SPIR, which is inspired by the PIR-balancing technique of [12]. The core idea is to divide the database into multiple subdatabases and let the client simultaneously retrieve each subdatabase. Concretely, the database DB is divided into n_r subdatabases for some $n_r \leq n$. Then the client uses the same index to retrieve an entry from each subdatabase. Among the received entries, the client outputs its desired entry. However, this violates the database privacy in SPIR, where the client can only obtain its retrieved entry. Now we show how to resolve this in the two-server setting in a statistical way.

Assume that $(a_{0,t}, a_{1,t})$ is the answer tuple for the t-th subdatabase. Let $\boldsymbol{a}_0 = (a_{0,1}, \ldots, a_{0,n_r})$ and $\boldsymbol{a}_1 = (a_{1,1}, \ldots, a_{1,n_r})$, where n_r is the number of the subdatabases. If the client wants only the j-th entry, instead of downloading all the answer tuples from the servers, the client takes $\boldsymbol{e}_j$ as input and seeks to securely compute the function $(\langle \boldsymbol{a}_0, \boldsymbol{e}_j \rangle, \langle \boldsymbol{a}_1, \boldsymbol{e}_j \rangle)$. To enable this computation to be completed in a single roundtrip, we consider the following equation.

$$\langle \boldsymbol{a}_0, \boldsymbol{e}_j \rangle = \langle (\boldsymbol{a}_0 \oplus \boldsymbol{\eta}_0) \oplus \boldsymbol{\eta}_0, \boldsymbol{e}_j \rangle = \langle \boldsymbol{a}_0 \oplus \boldsymbol{\eta}_0, \boldsymbol{e}_j \rangle \oplus \langle \boldsymbol{\eta}_0, \boldsymbol{e}_j \rangle.$$

If $\boldsymbol{\eta}_0$ is a random vector shared between the servers, then $\langle \boldsymbol{\eta}_0, \boldsymbol{e}_j \rangle$ can be computed in a single roundtrip using the same method as in our first multi-query scheme (since both servers know $\boldsymbol{\eta}_0$). Moreover, the left server can send $\boldsymbol{a}_0 \oplus \boldsymbol{\eta}_0$ (as part of the answer message) to the client, enabling the client to compute $\langle \boldsymbol{a}_0 \oplus \boldsymbol{\eta}_0, \boldsymbol{e}_j \rangle$. This does not violate the query privacy, as $\boldsymbol{\eta}_0$ is unknown to the client. As a result, the client can compute $\langle \boldsymbol{a}_0, \boldsymbol{e}_j \rangle$ using a single round of interaction. A similar discussion applies to the computation of $\langle \boldsymbol{a}_1, \boldsymbol{e}_j \rangle$.

Let $(\mathsf{BasicQuery}, \mathsf{BasicAnswer}, \mathsf{BasicExtract})$ be a single-round SPIR protocol. That is, for a database DB with n entries and a query $\alpha \in [\![n]\!]$, the protocol proceeds as follows.

1. The client runs $\mathsf{BasicQuery}(n, \alpha)$ to generate the query messages q_0, q_1.
2. The left server runs $\mathsf{BasicAnswer}(0, \mathsf{DB}, q_0)$ to obtain the answer a_0; and the right server runs $\mathsf{BasicAnswer}(1, \mathsf{DB}, q_1)$ to obtain the answer a_1.
3. Given the answers a_0, a_1, the client runs $\mathsf{BasicExtract}(n, \alpha, a_0, a_1)$ to obtain the desired output.

Our balanced SPIR construction is described below.

Construction 5.1: Balancing Standard SPIR

Public parameters: Let $\mathsf{DB} = \{\mathsf{DB}_i\}_{i \in [\![n]\!]}$ be the database with each $\mathsf{DB}_i \in \{0,1\}^l$. Let $n_r \in [n]$ be any integer and $n_c = n/n_r$. We treat $[\![n_c]\!]$ as the ring $\mathbb{Z}_{n_c}$. The servers rewrite the database as $\mathsf{DB} = (\mathsf{RB}_t)_{t \in [n_r]}$, where $\mathsf{RB}_t = \{\mathsf{DB}_{(t-1)n_c+i}\}_{i \in [\![n_c]\!]}$ is the t-th subdatabase. Let $\mathbb{A}_0$ be the answer space of the left server and $\mathbb{A}_1$ be the answer space of the right server in the basic SPIR protocol.

Shared randomness. The servers share the following randomness.

- Two random vectors $\boldsymbol{\eta}_0 \in \mathbb{A}_0^{n_r}, \boldsymbol{\eta}_1 \in \mathbb{A}_1^{n_r}$ and two random values $\bar{\eta} \in \mathbb{A}_0, \tilde{\eta} \in \mathbb{A}_1$.

Query. Let $\alpha \in [\![n]\!]$ be the index, then CL proceeds as follows.

1. Find $(\alpha_r, \alpha_c) \in [n_r] \times [\![n_c]\!]$ by computing $\alpha_c = \alpha \bmod n_c$ and $\alpha_r = (\alpha - \alpha_c)/n_c + 1$.
2. Compute the query message $(q_0, q_1) \leftarrow \mathsf{BasicQuery}(n_c, \alpha_c)$.
3. Sample a random $\boldsymbol{b}_0 \in \mathbb{F}_2^{n_r}$ and computes $\boldsymbol{b}_1 = \boldsymbol{e}_{\alpha_r} \oplus \boldsymbol{b}_0$, where $\boldsymbol{e}_{\alpha_r}$ is the α_r-th unit vector of length n_r.
4. Send $(q_0, \boldsymbol{b}_0)$ to LS and $(q_1, \boldsymbol{b}_1)$ to RS.

Answer.

1. LS computes $\boldsymbol{a}_0 = (a_{0,t})_{t \in [n_r]}$ with $a_{0,t} \leftarrow \mathsf{BasicAnswer}(\mathsf{RB}_t, q_0)$.
2. LS computes $\bar{\boldsymbol{a}}_0 = \boldsymbol{a}_0 \oplus \boldsymbol{\eta}_0, c_0 = \langle \boldsymbol{\eta}_0, \boldsymbol{b}_0 \rangle \oplus \bar{\eta}, d_0 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_0 \rangle \oplus \tilde{\eta}$.
3. LS sends $(\bar{\boldsymbol{a}}_0, c_0, d_0)$ to CL.
4. RS computes $\boldsymbol{a}_1 = (a_{1,t})_{t \in [n_r]}$ with each $a_{1,t} \leftarrow \mathsf{BasicAnswer}(\mathsf{RB}_t, q_1)$.
5. RS computes $\bar{\boldsymbol{a}}_1 = \boldsymbol{a}_1 \oplus \boldsymbol{\eta}_1, c_1 = \langle \boldsymbol{\eta}_0, \boldsymbol{b}_1 \rangle \oplus \bar{\eta}, d_1 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_1 \rangle \oplus \tilde{\eta}$.
6. RS sends $(\bar{\boldsymbol{a}}_1, c_1, d_1)$ to CL.

Extract. CL computes the desired output as follows.

1. Compute $a_0 = \langle \bar{\boldsymbol{a}}_0, \boldsymbol{e}_{\alpha_r} \rangle \oplus c_0 \oplus c_1$ and $a_1 = \langle \bar{\boldsymbol{a}}_1, \boldsymbol{e}_{\alpha_r} \rangle \oplus d_0 \oplus d_1$.
2. Compute $x \leftarrow \mathsf{BasicExtract}(n_c, \alpha_c, a_0, a_1)$.

Theorem 8. *If the basic SPIR protocol is statistically secure, then Construction 5.1 is a statistically secure SPIR protocol.*

Proof. We prove correctness, client privacy, left-server privacy, and right-server privacy, respectively.

Correctness. In the Extract procedure, we have

$$\begin{aligned} a_0 =& \langle \bar{\boldsymbol{a}}_0, \boldsymbol{e}_{\alpha_r} \rangle \oplus c_0 \oplus c_1 = \langle \bar{\boldsymbol{a}}_0, \boldsymbol{e}_{\alpha_r} \rangle \oplus \langle \boldsymbol{\eta}_0, \boldsymbol{b}_0 \rangle \oplus \langle \boldsymbol{\eta}_0, \boldsymbol{b}_1 \rangle \\ =& \langle \bar{\boldsymbol{a}}_0, \boldsymbol{e}_{\alpha_r} \rangle \oplus \langle \boldsymbol{\eta}_0, \boldsymbol{b}_0 \oplus \boldsymbol{b}_1 \rangle = \langle \bar{\boldsymbol{a}}_0, \boldsymbol{e}_{\alpha_r} \rangle \oplus \langle \boldsymbol{\eta}_0, \boldsymbol{e}_{\alpha_r} \rangle \\ =& \langle \bar{\boldsymbol{a}}_0 \oplus \boldsymbol{\eta}_0, \boldsymbol{e}_{\alpha_r} \rangle = \langle \boldsymbol{a}_0, \boldsymbol{e}_{\alpha_r} \rangle \\ =& a_{0,\alpha_r}. \end{aligned}$$

Similarly, we have $a_1 = a_{1,\alpha_r}$. By the correctness of the basic SPIR protocol, the output x is the α_c-th entry of the database RB_{α_r}, which is exactly $\mathsf{DB}_{(\alpha_r - 1)n_c + \alpha_c} = \mathsf{DB}_\alpha$.

Client privacy. We need to construct a simulator Sim such that given (α, x), Sim can output a simulated view that is statistically distinguishable from the real view of the client. Note that the real view is $(\alpha, (\bar{\boldsymbol{a}}_0, c_0, d_0), (\bar{\boldsymbol{a}}_1, c_1, d_1), x)$. Sim generates the simulated view as follows.

1. Use α to run the Query phase of Construction 5.1. Note that the simulation is perfect.
2. Use (α, x) to invoke the simulator of the basic SPIR scheme to output the simulated answers (a_0, a_1).
3. Sample two random vectors $\bar{\boldsymbol{a}}_0 = (\bar{a}_{0,1}, \ldots, \bar{a}_{0,n_r})$ and $\bar{\boldsymbol{a}}_1 = (\bar{a}_{1,1}, \ldots, \bar{a}_{1,n_r})$.
4. Sample two random values c_0, c_1 conditioned on $c_0 \oplus c_1 = \bar{a}_{0,\alpha_r} \oplus a_0$.
5. Sample two random values d_0, d_1 conditioned on $d_0 \oplus d_1 = \bar{a}_{1,\alpha_r} \oplus a_1$.
6. The simulated view is $(\alpha, (\bar{\boldsymbol{a}}_0, c_0, d_0), (\bar{\boldsymbol{a}}_1, c_1, d_1), x)$.

Now we show that the simulated and real views are statistically distinguishable. We consider the following hybrid distribution.

Hybird 0. We assume that a simulator Sim′ has the answers (a_0, a_1) from the servers, then it generates a view as follows.

1. Sample two random vectors $\bar{\boldsymbol{a}}_0 = (\bar{a}_{0,1}, \ldots, \bar{a}_{0,n_r})$ and $\bar{\boldsymbol{a}}_1 = (\bar{a}_{1,1}, \ldots, \bar{a}_{1,n_r})$.
2. Sample two random values c_0, c_1 subject to $c_0 \oplus c_1 = \bar{a}_{0,\alpha_r} \oplus a_0$.
3. Sample two random values d_0, d_1 subject to $d_0 \oplus d_1 = \bar{a}_{1,\alpha_r} \oplus a_1$.
4. Output $(\alpha, (\bar{\boldsymbol{a}}_0, c_0, d_0), (\bar{\boldsymbol{a}}_1, c_1, d_1), x)$.

We first show that the Hybird 0 is statistically indistinguishable from the real view. Note that in the real view, $\bar{\boldsymbol{a}}_0 = \boldsymbol{a}_0 \oplus \boldsymbol{\eta}_0, c_0 = \langle \boldsymbol{\eta}_0, \boldsymbol{b}_0 \rangle \oplus \bar{\eta}$ and $d_0 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_0 \rangle \oplus \tilde{\eta}$. Moreover, $\bar{\boldsymbol{a}}_1 = \boldsymbol{a}_1 \oplus \boldsymbol{\eta}_1, c_1 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_1 \rangle \oplus \bar{\eta}$ and $d_1 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_1 \rangle \oplus \tilde{\eta}$. Since $\boldsymbol{\eta}_0 = (\eta_{0,1}, \ldots, \eta_{0,n_r})$ and $\boldsymbol{\eta}_1 = (\eta_{1,1}, \ldots, \eta_{1,n_r})$ are random vectors that are unknown to the client, hence $\bar{\boldsymbol{a}}_0 = (\bar{a}_{0,1}, \ldots, \bar{a}_{0,n_r})$ and $\bar{\boldsymbol{a}}_1 = (\bar{a}_{1,1}, \ldots, \bar{a}_{1,n_r})$ are in fact random vectors. Moreover, note that c_0, c_1 are two random values subject to $c_0 \oplus c_1 = \eta_{0,\alpha_r} = \bar{a}_{0,\alpha_r} \oplus a_{0,\alpha_r}$, and d_0, d_1 are two random values subject to $d_0 \oplus d_1 = \eta_{1,\alpha_r} = \bar{a}_{1,\alpha_r} \oplus a_{1,\alpha_r}$. By the correctness of the protocol, we know that $a_{0,\alpha_r} = a_0$ and $a_{1,\alpha_r} = a_1$. Therefore, the distribution of the real view is identical to Hybird 0.

It remains to show that Hybird 0 and the simulated view are statistically indistinguishable. Note that these two views differ only in the answer tuple (a_0, a_1). In Hybird 0, (a_0, a_1) is the real answer tuple from the servers, while in the simulated view, (a_0, a_1) is simulated by the simulator for the underlying SPIR protocol. By the statistical security of the underlying SPIR protocol, we know that (a_0, a_1) in Hybird 0 is statistically indistinguishable from that in the simulated view. Consequently, Hybird 0 and the simulated view are statistically indistinguishable.

Left-server privacy. The left-server privacy is easy to see. Note the real view of the left server includes $(\mathsf{DB}, (q_0, \boldsymbol{v}_0))$. The query message q_0 can be simulated using the simulator of the basic SPIR protocol, and $\boldsymbol{v}_0$ is a random binary vector of length n_r and can be simulated perfectly.

Right-server privacy. The proof is the same as that for left-server privacy and we omit it. □

The complexity analysis is simple and we directly conclude it as follows.

Complexity. Let $S_r(n)$ be the randomness size. Let $S_q(n)$ and $T_q(n)$ be the query size and computational complexity of BasicQuery, respectively. Let $S_a(n)$ and $T_a(n)$ be the answer size and computational complexity of BasicAnswer. Let $T_e(n)$ be the computational complexity of BasicExtract. The complexity of Construction 5.1 is as follows.

- Randomness complexity. The size of the shared randomness is $O(n_r \cdot S_r(n_c) + 1)$.
- Query complexity. The query size is $S_q(n_c) + 2n_r$, and the computational complexity of Query is $T_q(n_c) + O(n_r)$.
- Answer complexity. The answer size is $n_r \cdot S_a(n_c) + 4l$, and the computational complexity of Answer is $n_r \cdot T_a(n_c) + O(ln_r)$.
- Extract complexity. The computational complexity of Extract is $T_e(n_r) + O(n_r)$.

5.2 Applying to SPIR with Preprocessing: Multi-Query SPIR with Sublinear Amortized Time and Client-Storage

We now extend our balancing technique to the preprocessing model. In particular, we consider applying it to our multi-query SPIR protocol. Recall that our protocol has linear offline communication and client-storage. Using our balancing technique, we can obtain a multi-query SPIR protocol with sublinear offline communication and client-storage.

Recall that the core idea is to divide the database DB into $n_r \in [n]$ subdatabases. Assume the database DB is divided into n_r subdatabases $\{\mathsf{RB}_t\}_{t \in [n_r]}$ with each RB_t being of size $n_c = n/n_r$. In the offline stage, the client generates the hint sets $S_1, \ldots, S_w$ with $w = (\ln 4) n_c^{1/2}$ and sends these sets to the left server. Then for each subdatabase RB_t, the left server computes the corresponding hint values $h_{t,1}, \ldots, h_{t,w}$ and sends them to the client. The client stores $(S_j, h_{1,j}, \ldots, h_{n_r,j})_{j \in [w]}$ as the hint table. Whenever the client wants to make a query $\alpha = (\alpha_r, \alpha_c)$, it must accomplish two tasks: retrieve an entry and request a new hint. The way of retrieving an entry is the same as in Construction 5.1. We only need to consider how to request a new hint. Note that in our original multi-query protocol, each hint is of the form (S_j, h_j), meaning that the hint set corresponds to a single hint value. To perform the refresh procedure, the client samples a new hint set and requests the corresponding hint value. However, in our balanced scheme, each hint is of the form $(S_j, h_{1,j}, \ldots, h_{n_r,j})$. That is, each hint set corresponds to multiple hint values. To generate a new hint, the client must request the hint values for all subdatabases using the same hint set. For the sake of completeness, we present the formal description of our final protocol below. Let β be the number of queries.

Protocol 5.2: Multi-Query SPIR with Sublinear Client-Storage

Shared randomness. The servers share the following randomness.

- n_r random vectors $\boldsymbol{r}_t = (r_{t,1}, \ldots, r_{t,w}) \in \mathcal{D}^w, t \in [n_r]$ with $w = (\ln 4) n_c^{1/2}$.
- $3\beta \cdot n_r$ random values $\{(r_{t,w+u}, \bar{r}_{t,u}, \tilde{r}_{t,u})\}_{t\in[n_r],u\in[\beta]}$ from $\mathcal{D}$. Each tuple $\{(r_{t,w+u}, \bar{r}_{t,u}, \tilde{r}_{t,u})\}_{t\in[n_r]}$ is used for the u-th query.
- Two random vectors $\boldsymbol{\eta}_0, \boldsymbol{\eta}_1 \in \mathcal{D}^{n_r}$ and two random values $\bar{\eta}, \tilde{\eta} \in \mathcal{D}$.

Offline stage

1. CL samples w random size-$n_c^{1/2}$ subsets $S_1, \ldots, S_w$ of $[\![n_c]\!]$.
2. CL sends $S_1, \ldots, S_w$ to LS.
3. LS computes $h_{t,j} = r_{t,j} \oplus \sum_{i\in S_j} \mathsf{RB}_{t,i}$ for all $t \in [n_r], j \in [w]$.
4. LS sends $\{h_{t,j}\}_{t\in[n_r],j\in[w]}$ to CL.
5. CL stores $\{(S_j, h_{1,j}, \ldots, h_{n_r,j})\}_{j\in[w]}$.
6. CL initializes the hint index-set $M = [w]$.

Online stage

1. Query. For the u-th query $\alpha = (\alpha_r, \alpha_c) \in [n_r] \times [\![n_c]\!]$, CL does the followings.
   (a) CL samples a single bit b, which equals 0 with probability $(n^{1/2}-1)/n$ and 1 with the remaining probability.
   (b) CL samples a random size-$(n_c^{1/2}-1)$ subset S of $[\![n_c]\!]\backslash\{\alpha_c\}$ and sets $S_{\mathsf{new}} = S \cup \{\alpha_c\}$.
   (c) If $b = 0$ or $\alpha_c \notin S_j$ for all $j \in M$, then CL computes $S_{\mathsf{query}} = S_{\mathsf{new}}$ and $k = 0$. Otherwise, CL finds the smallest $k \in M$ such that $\alpha_c \in S_k$ and sets $S_{\mathsf{query}} = S_k$.
   (d) CL computes $S^h = S_{\mathsf{new}}\backslash\{\alpha_c^*\}$ and $S^* = S_{\mathsf{query}}\backslash\{\alpha_c^*\}$, where α_c^* is sampled as follows.
      – If $b = 0$, α_c^* is a random element in $S_{\mathsf{new}}\backslash\{\alpha_c\}$.
      – If $b = 1$, $\alpha_c^* = \alpha_c$.
   (e) CL chooses random $\boldsymbol{v}_0 \in \mathbb{F}_2^{w+u}$ and computes $\boldsymbol{v}_1 = \boldsymbol{e}_{k+1}^{w+u} \oplus \boldsymbol{v}_0$, where $\boldsymbol{e}_{k+1}^{w+u}$ is the $(k+1)$-th length-$(w+u)$ unit vector.
   (f) CL samples a random $\boldsymbol{b}_0 \in \mathbb{F}_2^{n_r}$ and computes $\boldsymbol{b}_1 = \boldsymbol{e}_{\alpha_r}^{n_r} \oplus \boldsymbol{b}_0$, where $\boldsymbol{e}_{\alpha_r}^{n_r}$ is the α_r-th unit vector of length n_r.
   (g) CL sends $(S^h, \boldsymbol{v}_0, \boldsymbol{b}_0)$ to LS and $(S^*, \boldsymbol{v}_1, \boldsymbol{b}_1)$ to RS.
2. Answer.
   (a) For each $t \in [n_r]$, the two servers does the followings.
      i. Denote $(r_t, \bar{r}_t, \tilde{r}_t) = (r_{t,w+u}, \bar{r}_{t,u}, \tilde{r}_{t,u})$ and $\bar{\boldsymbol{r}}_t = (\bar{r}_t, \boldsymbol{r}_t)$.
      ii. LS computes $h_t = r_t \oplus \sum_{i\in S^h} \mathsf{RB}_{t,i}$ and $a_{0,t} = \langle \boldsymbol{v}_0, \bar{\boldsymbol{r}}_t \rangle \oplus \tilde{r}_t$.
      iii. RS computes $h_t^* = \sum_{i\in S^*} \mathsf{RB}_{t,i}$ and $a_{1,t} = \langle \boldsymbol{v}_1, \bar{\boldsymbol{r}}_t \rangle \oplus h_t^* \oplus \tilde{r}_t$.
      iv. LS and RS update $\boldsymbol{r}_t \leftarrow (\boldsymbol{r}_t, r_t)$.
   (b) LS computes $\boldsymbol{h} = (h_1, \ldots, h_{n_r}), \bar{\boldsymbol{a}}_0 = \boldsymbol{a}_0 \oplus \boldsymbol{\eta}_0, c_0 = \langle \boldsymbol{\eta}_0, \boldsymbol{b}_0 \rangle \oplus \bar{\eta}, d_0 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_0 \rangle \oplus \tilde{\eta}$, where $\boldsymbol{a}_0 = (a_{0,1}, \ldots, a_{0,n_r})$.
   (c) LS sends $(\boldsymbol{h}, \bar{\boldsymbol{a}}_0, c_0, d_0)$ to CL.
   (d) RS computes $\bar{\boldsymbol{a}}_1 = \boldsymbol{a}_1 \oplus \boldsymbol{\eta}_1, c_1 = \langle \boldsymbol{\eta}_0, \boldsymbol{b}_1 \rangle \oplus \bar{\eta}, d_1 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_1 \rangle \oplus \tilde{\eta}$, where $\boldsymbol{a}_1 = (a_{1,1}, \ldots, a_{1,n_r})$.
   (e) RS sends $(\bar{\boldsymbol{a}}_1, c_1, d_1)$ to CL.
3. Extract. CL computes the desired output as follows.
   – If $k = 0$, return $\perp$.
   – If $k > 0$, compute $a_0 = \langle \bar{\boldsymbol{a}}_0, \boldsymbol{e}_{\alpha_r} \rangle \oplus c_0 \oplus c_1, a_1 = \langle \bar{\boldsymbol{a}}_1, \boldsymbol{e}_{\alpha_r} \rangle \oplus d_0 \oplus d_1$ and return $x_\alpha = a_0 \oplus a_1 \oplus h_{\alpha_r,k}$.
4. Refresh. If $k > 0$, then the client performs the following procedures.
   (a) Parse $\boldsymbol{h} = (h_1, \ldots, h_{n_r})$ and compute $h_{w+u,t} = h_t \oplus x_\alpha$ for all $t \in [n_r]$ and $S_{w+u} = S_{\mathsf{new}}$.
   (b) Store the new hint $(S_{w+u}, h_{w+u,1}, \ldots, h_{w+u,n_r})$ with index $w + u$.
   (c) Update $M \leftarrow M \cup \{w + u\}\backslash\{k\}$.

Theorem 9. *Protocol 5.2 is a statistically secure two-server SPIR protocol.*

Proof (sketch). The detailed security proof can be obtained simply by combining the proofs for Theorems 7 and 8. We only give a sketch here.

Correctness. By the correctness of Protocol 4, we know that the client can recover RB_{t,α_c} from $(a_{0,t}, a_{1,t})$. Moreover, by the correctness of Construction 5.1, the client obtains $(a_0, a_1) = (a_{0,\alpha_r}, a_{1,\alpha_r})$. As a result, the output of the client will be $\mathsf{RB}_{\alpha_r,\alpha_c} = \mathsf{DB}_{(\alpha_r-1)n_c+\alpha_c} = \mathsf{DB}_\alpha$.

Client privacy. By the client privacy of Construction 5.1, we can guarantee that the client obtains only $(a_{0,\alpha_r}, a_{1,\alpha_r})$. By the client privacy of Protocol 4, the answer tuple $(a_{0,\alpha_r}, a_{1,\alpha_r})$ reveals only $\mathsf{RB}_{\alpha_r,\alpha_c} = \mathsf{DB}_\alpha$ to the client.

Left-server privacy. The left server receives $(S^h, \boldsymbol{v}_0, \boldsymbol{b}_0)$ from the client. Note that $\boldsymbol{v}_0, \boldsymbol{b}_0$ are two random binary vectors, leaking nothing about the query index. Moreover, by the left-server privacy of Protocol 4, S^h is a random size-$(n_c^{1/2} - 1)$ subset of $[\![n_c]\!]$, which leaks nothing about the query index.

Right-server privacy. The right server receives $(S^*, \boldsymbol{v}_1, \boldsymbol{b}_1)$ from the client. Note that $\boldsymbol{v}_1, \boldsymbol{b}_1$ are two random binary vectors, leaking nothing about the query index. Moreover, by the right-server privacy of Protocol 4, S^* is a random size-$(n_c^{1/2} - 1)$ subset of $[\![n_c]\!]$, which leaks nothing about the query index. □

Now we analyze the complexity of the above multi-query protocol. We defer the detailed analysis to Appendix D.4. Let l be the entry length, the complexity is as follows.

- Randomness complexity. The size of the shared randomness is $O(ln_r(n_c^{1/2} + \beta))$.
- Offline complexity. The computation and communication are $O(ln_r n_c + n_c \log n_c)$ and $O(n_c \log n_c + ln_r n_c^{1/2})$, respectively. Further, the client-storage is $O(n_c \log n_c + ln_r n_c^{1/2})$.
- Online complexity. For the u-th query, the total computation and communication are $O(n_c^{1/2} \log n_c + ln_r(n_c^{1/2} + u))$ and $O(n_c^{1/2} \log n_c + u + ln_r)$, respectively.

Parameter choices. Note that $n_r n_c = n$. To minimize the asymptotic client-storage, we choose $n_c = \Theta(n^{2/3})$ (so $n_r = \Theta(n^{1/3})$). This way, the client-storage is $O(n_c \log n_c + ln_r n_c^{1/2}) = \tilde{O}(ln^{2/3})$. Moreover, the randomness complexity is $O(ln^{2/3} + ln^{1/3}\beta))$. The offline computation and communication are $\tilde{O}(ln)$ and $\tilde{O}(ln^{2/3})$, respectively. And the online computation and communication of the u-th query are $\tilde{O}(ln^{2/3} + ln^{1/3}u)$ and $\tilde{O}(ln^{1/3} + u)$, respectively.

By letting the parties run the offline stage per $\Theta(n^{1/3})$ queries (i.e., $\beta = \Theta(n^{1/3})$), we can achieve $\tilde{O}(ln^{2/3})$ amortized time. Moreover, the size of the shared randomness is $O(ln^{2/3})$. And the online computation and communication per query are $\tilde{O}(ln^{2/3})$ and $\tilde{O}(ln^{1/3})$, respectively.

6 Conclusion

Cryptographic constructions with statistical security are of fundamental theoretical interest in maximizing security and minimizing assumptions. When considering a cryptographic primitive, the initial inquiry would be whether a statistical solution exists. In the context of PIR, prior works have showed that statistical (two-server) PIR with symmetric privacy *or* sublinear online time exists. Our work confirms the existence of information-theoretic two-server PIR with both symmetric privacy *and* sublinear online time. Our protocols rely on an additional assumption that the servers share a pragmatic level of private randomness, which has been proved inevitable for achieving statistical security in two-server SPIR, as showed by Gertner et al. [25].

References

1. Dawex: Sell, buy and share data. https://www.dawex.com/en/
2. Quodd: Market data on demand. https://www.quodd.com/
3. Ambainis, A.: Upper bound on the communication complexity of private information retrieval. In: International Colloquium on Automata, Languages, and Programming. pp. 401–407 (1997)
4. Beimel, A., Ishai, Y.: Information-theoretic private information retrieval: A unified construction. In: ICALP 2001. pp. 912–926. Springer (2001)
5. Beimel, A., Ishai, Y., Kushilevitz, E., Malkin, T.: One-way functions are essential for single-server private information retrieval. In: STOC 1999. pp. 89–98 (1999)
6. Beimel, A., Ishai, Y., Kushilevitz, E., Raymond, J.F.: Breaking the o (n/sup 1/(2k-1)/) barrier for information-theoretic private information retrieval. In: FOCS 2002. pp. 261–270. IEEE (2002)
7. Beimel, A., Ishai, Y., Malkin, T.: Reducing the servers computation in private information retrieval: Pir with preprocessing. In: CRYPTO 2000. pp. 55–73 (2000)
8. Beimel, A., Ishai, Y., Malkin, T.: Reducing the servers' computation in private information retrieval: Pir with preprocessing. Journal of Cryptology pp. 125–151 (2004)

9. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing: Improvements and extensions. In: CCS 2016. pp. 1292–1303 (2016)
10. Cachin, C., Micali, S., Stadler, M.: Computationally private information retrieval with polylogarithmic communication. In: EUROCRYPT 1999. pp. 402–414 (1999)
11. Chang, Y.: Single database private information retrieval with logarithmic communication. In: ACISP 2004. pp. 50–61 (2004)
12. Chor, B., Goldreich, O., Kushilevitz, E., Sudan, M.: Private information retrieval. In: FOCS 1995. pp. 41–50 (1995)
13. Chor, B., Gilboa, N.: Computationally private information retrieval. In: STOC 1997. pp. 304–313 (1997)
14. Chor, B., Kushilevitz, E.: A zero-one law for boolean privacy. In: STOC 1989. pp. 62–72 (1989)
15. Corrigan-Gibbs, H., Henzinger, A., Kogan, D.: Single-server private information retrieval with sublinear amortized time. In: EUROCRYPT 2022. pp. 3–33 (2022)
16. Corrigan-Gibbs, H., Kogan, D.: Private information retrieval with sublinear online time. In: EUROCRYPT 2020. pp. 44–75 (2020)
17. Di Crescenzo, G., Malkin, T., Ostrovsky, R.: Single database private information retrieval implies oblivious transfer. In: EUROCRYPT 2000. pp. 122–138 (2000)
18. Diffie, W., Hellman, M.E.: New directions in cryptography. IEEE Trans. Inf. Theory pp. 644–654 (1976)
19. Döttling, N., Garg, S., Ishai, Y., Malavolta, G., Mour, T., Ostrovsky, R.: Trapdoor hash functions and their applications. In: CRYPTO 2019. pp. 3–32 (2019)
20. Dvir, Z., Gopi, S.: 2-server pir with subpolynomial communication. Journal of the ACM (JACM) pp. 1–15 (2016)
21. Efremenko, K.: 3-query locally decodable codes of subexponential length. In: STOC 2009. pp. 39–44 (2009)
22. Fisch, B., Lazzaretti, A., Liu, Z., Papamanthou, C.: Thorpir: Single server PIR via homomorphic thorp shuffles. In: CCS 2024. pp. 1448–1462 (2024)
23. Freedman, M.J., Ishai, Y., Pinkas, B., Reingold, O.: Keyword search and oblivious pseudorandom functions. In: TCC 2005. pp. 303–324 (2005)
24. Gentry, C., Ramzan, Z.: Single-database private information retrieval with constant communication rate. In: ICALP 2005. pp. 803–815. Springer (2005)
25. Gertner, Y., Ishai, Y., Kushilevitz, E., Malkin, T.: Protecting data privacy in private information retrieval schemes. In: STOC 1998. pp. 151–160 (1998)
26. Ghoshal, A., Zhou, M., Shi, E.: Efficient pre-processing pir without public-key cryptography. In: EYROCRYPT 2024. pp. 210–240. Springer (2024)
27. Ghoshal, A., Zhou, M., Shi, E., Peng, B.: Pseudorandom functions with weak programming privacy and applications to private information retrieval. In: EUROCRYPT 2025 (2025)
28. Hoare, C.A.R.: Algorithm 64: quicksort. Communications of the ACM p. 321 (1961)
29. Hoover, A., Patel, S., Persiano, G., Yeo, K.: Plinko: Single-server PIR with efficient updates via invertible prfs. In: EUROCRYPT 2025 (2025)
30. Ishai, Y., Shi, E., Wichs, D.: PIR with client-side preprocessing: Information-theoretic constructions and lower bounds. CRYPTO 2024 (2024)
31. Kogan, D., Corrigan-Gibbs, H.: Private blocklist lookups with checklist. In: USENIX Security 21. pp. 875–892 (2021)
32. Kushelvitz, E.: Privacy and communication complexity. SIAM Journal on Discrete Mathematics pp. 273–284 (1992)
33. Kushilevitz, E., Ostrovsky, R.: Replication is not needed: Single database, computationally-private information retrieval. In: FOCS 1997. pp. 364–373 (1997)
34. Kushilevitz, E., Ostrovsky, R.: One-way trapdoor permutations are sufficient for non-trivial single-server private information retrieval. In: EUROCRYPT 2000. pp. 104–121 (2000)
35. Lazzaretti, A., Papamanthou, C.: Near-optimal private information retrieval with preprocessing. In: TCC 2023. pp. 406–435. Springer (2023)
36. Lazzaretti, A., Papamanthou, C.: Treepir: Sublinear-time and polylog-bandwidth private information retrieval from ddh. In: CRYPTO 2023. pp. 284–314. Springer (2023)
37. Lazzaretti, A., Papamanthou, C.: Single pass client-preprocessing private information retrieval. USENIX Security 2024 (2024)
38. Mingxun, Z., PARK, A., SHI, E., et al.: Piano: Extremely simple, single-server pir with sublinear server computation [c]. In: SP 2024 (2024)
39. Ren, L., Mughees, M.H., Sun, I.: Simple and practical amortized sublinear private information retrieval using dummy subsets. In: CCS 2024. pp. 1420–1433 (2024)
40. Shi, E., Aqeel, W., Chandrasekaran, B., Maggs, B.: Puncturable pseudorandom sets and private information retrieval with near-optimal online bandwidth and time. In: CRYPTO 2021. pp. 641–669 (2021)
41. Singh, J., Wei, Y., Zikas, V.: Information-theoretic multi-server private information retrieval with client preprocessing. In: TCC 2024. pp. 423–450 (2024)

42. Wang, Z., Ren, L.: Single-server client preprocessing PIR with tight space-time trade-off. IACR Cryptol. ePrint Arch. p. 1845 (2024)
43. Yekhanin, S.: Towards 3-query locally decodable codes of subexponential length. Journal of the ACM (JACM) pp. 1–16 (2008)
44. Zhou, M., Lin, W.K., Tselekounis, Y., Shi, E.: Optimal single-server private information retrieval. In: EUROCRYPT 2023. pp. 395–425 (2023)

A Proofs for Our First Single-Query SPIR Protocol

A.1 Correctness Proof

We would like to prove that the protocol is correct with probability at least $1/2$. This way, by running λ copies, the client can obtain the correct output with probability at least $1-2^{\lambda}$. Namely, the failure probability is $2^{-\lambda}$.

Proof. We first show that the query will succeed if $b=1$ and $\alpha-\delta_k \in S$ for some $k \in [w]$. Without loss of generality, let (δ_k, h_k) be the consumed hint, where $h_k = r_k + \sum_{i\in S_k} \mathsf{DB}_i$ and $S_k = \{i+\delta_k\}_{i\in S}$. We need to show that $a_0 + a_1 + h_k = \mathsf{DB}_\alpha$. Notice that

$$\begin{aligned}
&a_0 + a_1 + h_k \\
=&\hat{r}\cdot\alpha_0 + \tilde{r} + \hat{r}\cdot\alpha_1 - r_k - h^* - \tilde{r} + r_k + \sum_{i\in S_k} \mathsf{DB}_i \\
=&\hat{r}\cdot(\alpha_0+\alpha_1) - \sum_{i\in S^*} \mathsf{DB}_i + \sum_{i\in S_k} \mathsf{DB}_i \\
=&\hat{r}\cdot(\alpha-\alpha^*) + \mathsf{DB}_{\alpha^*}.
\end{aligned}$$

By the protocol, we know that $\alpha^* = \alpha$ when $b=1$, which implies that $a_0 + a_1 + h_k = \mathsf{DB}_\alpha$.

We now prove that the probability that $b=1$ and $\alpha-\delta_k \in S$ for some $k\in[w]$ is at least $1/2$. This way, our protocol guarantees that the client will obtain the desired output with a probability of at least $1/2$. Denote E_1 the event that $b=0$ and E_2 the event that $\alpha-\delta_j \notin S$ for all $j\in[w]$. We need to show that $\Pr(E_1\cup E_2)\le 1/2$.

First, note that $\Pr(E_1) = (n^{1/2}-1)/n$. Since $(n^{1/2}-1)/n$ reaches its maximum at $n=4$, we have $\Pr(E_1)\le 1/4$. Moreover, since S is a size-$n^{1/2}$ subset of $[\![n]\!]$ and each δ_j is a random element of $[\![n]\!]$, we know that $\Pr(E_2) = (1-n^{1/2}/n)^w = (1-n^{-1/2})^w \le e^{-w/n^{1/2}}$. Note that $w = (\ln 4)n^{1/2}$, hence $\Pr(E_2)\le e^{-\ln 4} = 1/4$. Therefore, we have $\Pr(E_1\cup E_2)\le \Pr(E_1)+\Pr(E_2)\le 1/2$. This completes the proof. □

A.2 Client Privacy Proof

We show that the client obtains only the retrieved value if the query succeeds and nothing otherwise.

Proof. We need to show that there exists a simulator Sim such that, using the input and output of the client, Sim can generate a simulated view that is statistically indistinguishable from the real view of the client. The simulator Sim works as follows.

Simulated offline stage.

1. Sample a random size-$n^{1/2}$ subset S of $[\![n]\!]$ and w random shifts $\delta_1,\dots,\delta_w \in [\![n]\!]$.
2. Sample w random values $h_1,\dots,h_w \in \mathbb{F}$ and simulate the left server sending $\{h_j\}_{j\in[w]}$ to the client.
3. Store $\{h_j\}_{j\in[w]}$.

Simulated online stage.

- If the output of the client is $x_\alpha \neq \perp$,
  1. Choose some $k\in[w]$ and sample two random values $a_0, a_1 \in \mathbb{F}$ subject to $a_0+a_1 = x_\alpha - h_k$.
  2. Simulate the left server sending a_0 to the client; simulate the right server sending a_1 to the client.
- If the output of the client is $\perp$,

1. Sample two random values $a_0, a_1 \in \mathbb{F}$.
2. Simulate the left server sending a_0 to the client; simulate the right server sending a_1 to the client.

Now, we show that the real view and simulated view are statistically indistinguishable. In the real view, the client receives w hint values $\{h_j\}_{j\in[w]}$ from the left server during the offline stage with each $h_j = r_j + \sum_{i\in S_j} \mathsf{DB}_i$. Since each $r_j \in \mathbb{F}$ is a random value that is unknown to the client, h_j is also a random value. As a result, the simulation for the offline stage is perfect. In the online stage, we consider two cases: the query succeeds or fails.

1. *When the query succeeds*, the client receives $a_0 = \hat{r} \cdot \alpha_0 + \tilde{r}$ from the left server and $a_1 = \hat{r} \cdot \alpha_1 - r_k - h^* - \tilde{r}$ from the right server. Since $\tilde{r}$ is a random value, the client in fact receives two random values a_0, a_1 subject to

$$a_0 + a_1 = (\hat{r} \cdot \alpha_0 + \tilde{r}) + (\hat{r} \cdot \alpha_1 - r_k - h^* - \tilde{r}) = \hat{r} \cdot (\alpha_0 + \alpha_1) - r_k - h^*.$$

Since the query succeeds, we know that $\alpha_0 + \alpha_1 = \mathsf{lift}(\alpha - \alpha^*) = 0$ and $h^* = \sum_{i\in S^*} \mathsf{DB}_i = \sum_{i\in S_k\setminus\{\alpha\}} \mathsf{DB}_i$. Hence

$$a_0 + a_1 = -r_k - \sum_{i\in S_k\setminus\{\alpha\}} \mathsf{DB}_i = \mathsf{DB}_\alpha - h_k.$$

That is, in the real view, a_0, a_1 are in fact two random values conditioned on that $a_0 + a_1 = \mathsf{DB}_\alpha - h_k$. Since $x_\alpha = \mathsf{DB}_\alpha$, the simulation for a_0, a_1 is perfect.

2. *When the query fails*, the client receives $a_0 = \hat{r} \cdot \alpha_0 + \tilde{r}$ from the left server. Further, the client receives a_1 from the right server, where a_1 is a random value if $k = 0$ and $\hat{r} \cdot \alpha_1 - r_k - h^* - \tilde{r}$ otherwise. Now we show that a_1 is always statistically indistinguishable from a random value, regardless of the exact value of k. When $k = 0$, a_1 is clearly random. We now consider the case that $k > 0$ (since the query fails, this implies that $b = 0$). Since $\tilde{r}$ is a random value, we know that a_0, a_1 are in fact two random values subject to

$$a_0 + a_1 = (\hat{r} \cdot \alpha_0 + \tilde{r}) + (\hat{r} \cdot \alpha_1 - r_k - h^* - \tilde{r}) = \hat{r} \cdot (\alpha_0 + \alpha_1) - r_k - h^*.$$

Since $k > 0$ and $b = 0$, we know that $\alpha_0 + \alpha_1 = \mathsf{lift}(\alpha - \alpha^*) \neq 0$. Note that $\hat{r}$ is a random non-zero field element, which means that $a_0 + a_1 = r - r_k - h^*$ with r being a random non-zero field element. Although $a_0 + a_1$ is not a uniformly random field element (due to $r \neq 0$), it is statistically indistinguishable from a random field element because $\log|\mathbb{F}| \geq \lambda$. As a result, we know that a_0, a_1 are statistically indistinguishable from the simulated a_0, a_1 (which are two random values). □

A.3 Left-Server Privacy Proof

Proof. For the offline stage, the left server receives a random size-$n^{1/2}$ subset S of $[\![n]\!]$ and w random shifts $\delta_1, \ldots, \delta_w \in [\![n]\!]$. This can be simulated perfectly, as the client has no input in the offline stage. In the online stage, the left server receives only a random value α_0, which can be simulated perfectly. □

A.4 Right-Server Privacy Proof

Proof. The right server receives nothing from the client during the offline stage. In the online stage, the right server receives $\perp$ if $k = 0$ and (S^*, k, α_1) otherwise. Note that $k = 0$ means that $\alpha - \delta_j \notin S$ for all $j \in [w]$. Moreover, $k > 0$ means that $\alpha - \delta_j \notin S$ for all $j < k$ and $\alpha - \delta_k \in S$. Since each δ_j is a random index, we know that the value of k is independent of α. As a result, the simulator can simulate k perfectly as follows.

1. Sample a random size-$n^{1/2}$ subset S of $[\![n]\!]$ and w random shifts $\delta_1, \ldots, \delta_w \in [\![n]\!]$.
2. If $1 - \delta_j \notin S$ for all $j \in [w]$, then set $k = 0$. Otherwise, find the smallest $k \in [w]$ subject to $1 - \delta_k \in S$.

Given the simulated k, the simulator can simulate the received message for the right server:

1. For $k = 0$, simulate the client sending $\perp$ to the right server.

2. For $k > 0$, sample a random size-$(n^{1/2}-1)$ subset S^* of $[\![n]\!]$ and a random value $\alpha_1 \in \mathbb{F}$, and simulate the client sending (S^*, k, α_1) to the right server.

We now show that the simulated view and the real one are statistically indistinguishable. First, we can easily show that the distribution of k in the simulated view is the same as that in the real view. For the case that $k = 0$, the right server receives $\perp$ in both the simulated and real views. When $k > 0$, the right server receives (S^*, k, α_1), where α_1 is a random value chosen by the client. To show that the simulation is perfect, we still need to show that S^* is a random size-$(n^{1/2}-1)$ subset S^* of $[\![n]\!]$.

Note that $S^* = S_{\mathsf{query}} \backslash \{\alpha^*\}$ with $\alpha \in S_{\mathsf{query}}$. We first show that $S_{\mathsf{query}} \backslash \{\alpha\}$ is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!] \backslash \{\alpha\}$. We have the following facts.

- When the query succeeds, we have $S_{\mathsf{query}} = \{i + \delta_k\}_{i \in S}$ with $\alpha - \delta_k \in S$. Let $s = \alpha - \delta_k$, then we have $S_{\mathsf{query}} \backslash \{\alpha\} = \{\alpha + i - s\}_{i \in S \backslash \{s\}}$. Since $\delta_k \in [\![n]\!]$ is random, we know that s is a random index subject to $s \in S$. Namely, s is a random index in S.
- When the query fails, we have $S_{\mathsf{query}} = \{\alpha + i - s\}_{i \in S}$ with s being a random index in S. Moreover, $S_{\mathsf{query}} \backslash \{\alpha\} = \{\alpha + i - s\}_{i \in S \backslash \{s\}}$.

That is, whether the query succeeds or not, we have $S_{\mathsf{query}} \backslash \{\alpha\} = \{\alpha + i - s\}_{i \in S \backslash \{s\}}$ with s being a random index in S. Since S is a random size-$n^{1/2}$ subset of $[\![n]\!]$ and s is a random index in S, we know that $\{i - s\}_{i \in S \backslash \{s\}}$ is a random size-$(n^{1/2}-1)$ subset of $[n-1]$, which implies that $S_{\mathsf{query}} \backslash \{\alpha\}$ is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!] \backslash \{\alpha\}$.

We are now ready to show that S^* is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!]$. For any $T \subseteq [\![n]\!]$ subject to $|T| = n^{1/2} - 1$, then we have

$$\begin{aligned}
&\Pr(S^* = T) \\
=&\Pr(S_{\mathsf{query}} \backslash \{\alpha^*\} = T) \\
=&\Pr(b = 1) \cdot \Pr(S_{\mathsf{query}} \backslash \{\alpha\} = T | b = 1) + \Pr(b = 0) \cdot \Pr(S_{\mathsf{query}} \backslash \{\alpha^*\} = T | b = 0).
\end{aligned}$$

We first consider the case that $\alpha \in T$. Note that if $b = 1$, then $\alpha^* = \alpha$, which implies that $\alpha \notin S_{\mathsf{query}} \backslash \{\alpha^*\}$. Since $\alpha \in T$, we have $\Pr(S_{\mathsf{query}} \backslash \{\alpha\} = T | b = 1) = 0$. Therefore, we have

$$\Pr(S^* = T) = \Pr(b = 0) \cdot \Pr(S_{\mathsf{query}} \backslash \{\alpha, \alpha^*\} = T \backslash \{\alpha\} | b = 0).$$

Note that if $b = 0$, then α^* is a random index in $S_{\mathsf{query}} \backslash \{\alpha\}$. Hence $S_{\mathsf{query}} \backslash \{\alpha, \alpha^*\}$ is a random size-$(n^{1/2}-2)$ subset of $[\![n]\!] \backslash \{\alpha\}$, which implies that

$$\Pr(S_{\mathsf{query}} \backslash \{\alpha, \alpha^*\} = T \backslash \{\alpha\} | b = 0) = \tbinom{n-1}{n^{1/2}-2}^{-1}.$$

Therefore, we have

$$\Pr(S^* = T) = \frac{n^{1/2} - 1}{n} \cdot \tbinom{n-1}{n^{1/2}-2}^{-1} = \tbinom{n}{n^{1/2}-1}^{-1}.$$

Now we consider the case that $\alpha \notin T$. Note that if $b = 0$, then $\alpha^* \neq \alpha$, which implies that $\alpha \in S_{\mathsf{query}} \backslash \{\alpha^*\}$. Since $\alpha \notin T$, we have $\Pr(S_{\mathsf{query}} \backslash \{\alpha\} = T | b = 1) = 0$. Therefore, we have

$$\begin{aligned}
&\Pr(S^* = T) \\
=&\Pr(b = 1) \cdot \Pr(S_{\mathsf{query}} \backslash \{\alpha\} = T | b = 1) \\
=&(1 - \frac{n^{1/2} - 1}{n}) \cdot \tbinom{n-1}{n^{1/2}-1}^{-1} \\
=&\tbinom{n}{n^{1/2}-1}^{-1}.
\end{aligned}$$

Based on the above discussion, we know that S^* is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!]$. This implies that the simulation is perfect. □

B Proofs for Our Second Single-Query SPIR Protocol

B.1 Correctness Proof

We prove that the protocol is correct with a probability of at least 1/2.

Proof. We first show that the query will succeed if $b = 1$ and $\alpha - \delta_k \in S$ for some $k \in [w]$. Let (δ_k, h_k) be the consumed hint, where $h_k = r_k \oplus \sum_{i \in S_k} \mathsf{DB}_i, S_k = \{i + \delta_k\}_{i \in S}$. We need to show that $a_0 \oplus a_1 \oplus h_k = \mathsf{DB}_\alpha$. Note that

$$\begin{aligned} & a_0 \oplus a_1 \oplus h_k \\ =&(\langle \boldsymbol{v}_0, \boldsymbol{r} \rangle \oplus \tilde{r}) \oplus (\langle \boldsymbol{v}_1, \boldsymbol{r} \rangle \oplus h^* \oplus \tilde{r}) \oplus h_k \\ =&\langle \boldsymbol{v}_0 \oplus \boldsymbol{v}_1, \boldsymbol{r} \rangle \oplus h^* \oplus h_k \\ =&\langle \boldsymbol{e}_{k+1}^{w+1}, \boldsymbol{r} \rangle \oplus h^* \oplus h_k. \end{aligned}$$

Note that $\boldsymbol{r} = (\bar{r}, r_1, \ldots, r_w)$ and $k \in [w]$, we have $\langle \boldsymbol{e}_{k+1}^{w+1}, \boldsymbol{r} \rangle = r_k$, implying that

$$a_0 \oplus a_1 \oplus h_k = r_k \oplus \sum_{i \in S^*} \mathsf{DB}_i \oplus r_k \oplus \sum_{i \in S_k} \mathsf{DB}_i = \sum_{i \in S^*} \mathsf{DB}_i \oplus \sum_{i \in S_k} \mathsf{DB}_i.$$

Since $S^* = S_{\mathsf{query}} \backslash \{\alpha\} = S_k \backslash \{\alpha\}$ if $b = 1$, we have $a_0 \oplus a_1 \oplus h_k = \mathsf{DB}_\alpha$.

It remains to show that the probability that $b = 1$ and $\alpha - \delta_k \in S$ for some $k \in [w]$ is at least $1/2$, which has been proved in the correctness proof for our first single-query protocol. □

B.2 Client Privacy Proof

Proof. We need to show that the simulator Sim taking the input and output of the client can generate a simulated view that is statistically indistinguishable from the real view of the client. our simulator works as follows.

Simulated offline stage.

1. Sample w random values $h_1, \ldots, h_w \in \mathcal{D}$ and simulate the left server sending $\{h_j\}_{j \in [w]}$ to the client.
2. Store $\{h_j\}_{j \in [w]}$.

Simulated online stage.

- If the output of the client is $x_\alpha \neq \perp$,
  1. Choose some $k \in [w]$ and sample two random values $a_0, a_1 \in \mathcal{D}$ subject to $a_0 \oplus a_1 = x_\alpha \oplus h_k$.
  2. Simulate the left server sending a_0 to the client; simulate the right server sending a_1 to the client.
- If the output of the client is $\perp$,
  1. Sample two random values $a_0, a_1 \in \mathcal{D}$.
  2. Simulate the left server sending a_0 to the client; simulate the right server sending a_1 to the client.

Now, we show that the real view and the simulated one are statistically indistinguishable. In the real view, the client receives w hint values $\{h_j\}_{j \in [w]}$ from the left server during the offline stage with each $h_j = r_j \oplus \sum_{i \in S_j} \mathsf{DB}_i$. Since each $r_j \in \mathcal{D}$ is a random value that is unknown to the client, h_j is also a random value. As a result, the simulation for the offline stage is perfect.

In the online stage, the client receives a_0 from the left server and a_1 from the right server, where $a_0 = \langle \boldsymbol{v}_0, \boldsymbol{r} \rangle \oplus \tilde{r}, a_1 = \langle \boldsymbol{v}_1, \boldsymbol{r} \rangle \oplus h^* \oplus \tilde{r}$. We consider two cases: the query succeeds or fails.

1. *When the query succeeds*, by the correctness of our protocol, we have $a_0 \oplus a_1 \oplus h_k = \mathsf{DB}_\alpha$ for some $k \in M$. Since $\tilde{r}$ is a random value, we know a_0 and a_1 are in fact two random values conditioned on $a_0 \oplus a_1 = \mathsf{DB}_\alpha \oplus h_k$. Therefore, our simulation is perfect.

1. *When the query fails*, notice that

$$\begin{aligned} & a_0 \oplus a_1 \\ =&(\langle \boldsymbol{v}_0, \boldsymbol{r} \rangle \oplus \tilde{r}) \oplus (\langle \boldsymbol{v}_1, \boldsymbol{r} \rangle \oplus h^* \oplus \tilde{r}) \\ =&\langle \boldsymbol{v}_0 \oplus \boldsymbol{v}_1, \boldsymbol{r} \rangle \oplus h^* \\ =&\langle \boldsymbol{e}_{k+1}^{w+1}, \boldsymbol{r} \rangle \oplus h^*. \end{aligned}$$

Here $\boldsymbol{r} = (\bar{r}, r_1, \ldots, r_w)$. If the query fails, we have $k = 0$, which implies that $\langle \boldsymbol{e}_{k+1}^{w+1}, \boldsymbol{r} \rangle = \langle \boldsymbol{e}_1^{w+1}, \boldsymbol{r} \rangle = \bar{r}$. Since $\bar{r}$ is a random value, $a_0 \oplus a_1 = \bar{r} \oplus h^*$ is also a random value. That is, a_0, a_1 are in fact two random values, hence our simulation for a_0, a_1 is perfect. This completes the client privacy proof. □

B.3 Left-Server Privacy Proof

Proof. In the offline stage, the left server receives a random size-$n^{1/2}$ subset S of $[\![n]\!]$ and w random shifts $\delta_1, \ldots, \delta_w \in [\![n]\!]$. This can be simulated perfectly, as the client has no input in the offline stage. In the online stage, the left server receives only a random binary vector $\boldsymbol{v}_0$, which can be simulated perfectly. □

B.4 Right-Server Privacy Proof

Proof. The right server receives nothing from the client during the offline stage. In the online stage, the right server receives the query message $(S^*, \boldsymbol{v}_1)$. Note that $\boldsymbol{v}_1$ is a random binary vector and can be simulated perfectly. As for the set S^*, if we can show that it is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!]$, then the simulator can simulate it perfectly. In fact, this has been proven in Appendix A.4, given that the distribution of S^* in our second single-query scheme is the same as that in our first single-query scheme. □

C Proofs for Our Multi-Query SPIR Protocol with Linear Client-Storage

Throughout this section, we use M_u to denote the hint index-set before the u-th query for each $u \in [\beta]$. As the basis of our security analysis, we first prove the following lemma.

Lemma 1. *For any $u \in [\beta]$, the hints $\{(S_j, h_j)\}_{j \in M_u}$ have the same distribution as $\{(S_j, h_j)\}_{j \in M_{u+1}}$.*

Proof. Notice that the size of each set M_u is always w. The reason is that the initial set $M_1 = [w]$ is of size w. If the query succeeds, then we take an element out and put a new one in; If the query fails, the set remains unchanged. Now, we show that the hints in two adjacent sets M_u and M_{u+1} are of the same distribution.

Let the hints in M_u be $\{(S_j, h_j)\}_{j \in M_u}$ and α be the u-th query index. If the query fails, then $M_{u+1} = M_u$ and the hints in M_{u+1} are still $\{(S_j, h_j)\}_{j \in M_u}$. If the query succeeds, let (S_k, h_k) be the consumed hint, then the hints in M_{u+1} are $\{(S_j, h_j)\}_{j \in M_{u+1}}$, where $M_{u+1} = M_u \cup \{w+u\} \backslash \{k\}$ and (S_{w+u}, h_{w+u}) is the new hint. Let $(k_0, k_1) = (k, m+u)$ if the query succeeds, and otherwise, let $(k_0, k_1) = (j_0, j_0)$ for some $j_0 \in M_u$ (e.g., the smallest element in M_u). Then we can say that the hints in M_u are

$$(\{(S_j, h_j)\}_{j \in M_u \backslash \{k_0\}}, (S_{k_0}, h_{k_0}))$$

and the hints in M_{u+1} are

$$(\{(S_j, h_j)\}_{j \in M_{u+1} \backslash \{k_1\}}, (S_{k_1}, h_{k_1})).$$

Note that each h_j is fully determined by (DB, S_j). To show that the hints in M_u and M_{u+1} have the same distribution, we only need to prove

$$(\mathsf{DB}, \{S_j\}_{j \in M_u \backslash \{k_0\}}, S_{k_0}) \equiv (\mathsf{DB}, \{S_j\}_{j \in M_{u+1} \backslash \{k_1\}}, S_{k_1}).$$

Note that it always holds that $M_u \backslash \{k_0\} = M_{u+1} \backslash \{k_1\}$. Moreover, both S_{k_0} and S_{k_1} are independent of other sets, hence we only need to show that S_{k_0} and S_{k_1} have the same distribution.

Let E_1 be the event that $b = 1$ and E_2 be the event that $\alpha \in S_k$ for some $k \in M_u$, then $E = E_1 \cap E_2$ is the event that the query succeeds. We first consider the distribution of S_{k_0}. For any $T \subseteq [\![n]\!]$ subject to $|T| = n^{1/2}$, we have

$$\Pr(S_{k_0} = T) = \Pr((S_k = T) \cap E) + \Pr((S_{j_0} = T) \cap \bar{E}).$$

For the case that $\alpha \notin T$, since $\alpha \in S_k$ when the query succeeds, we know that $\Pr((S_k = T) \cap E) = 0$. Therefore, we have

$$\Pr(S_{k_0} = T) = \Pr((S_{j_0} = T) \cap \bar{E}).$$

Now we consider the case that $\alpha \in T$. Denote $M_u = \{j_1, \ldots, j_w\}$ and for each $i \in [w]$, let E^i be the event that $\alpha \notin S_{j_d}$ for all $d < i$ and $\alpha \in S_{j_i}$. By our protocol, when E^i happens, the client will use

the j_i-th hint, i.e., $k = j_i$. Then we have

$$\begin{aligned}
&\Pr(S_{k_0} = T)\\
=&\Pr((S_k = T) \cap E) + \Pr((S_{j_0} = T) \cap \bar{E})\\
=&\Pr(E_1) \cdot \Pr((S_k = T) \cap E_2) + \Pr((S_{j_0} = T) \cap \bar{E})\\
=&\Pr(E_1) \cdot \sum_{i\in[w]} \Pr((S_{j_i} = T) \cap E^i) + \Pr((S_{j_0} = T) \cap \bar{E}).\\
=&\Pr(E_1) \cdot \sum_{i\in[w]} \Pr(E^i) \cdot \Pr(S_{j_i} = T|E^i) + \Pr((S_{j_0} = T) \cap \bar{E}).
\end{aligned}$$

Since $\alpha \in T$, we have

$$\Pr(S_{j_i} = T|E^i) = \Pr(S_{j_i}\backslash\{\alpha\} = T\backslash\{\alpha\}|E^i).$$

Note that each $S_{j_i}\backslash\{\alpha\}$ is a random size-$(n^{1/2} - 1)$ subset of $[\![n]\!]\backslash\{\alpha\}$. Since $T\backslash\{\alpha\} \subseteq [\![n]\!]\backslash\{\alpha\}$, we know that $\Pr(S_{j_i}\backslash\{\alpha\} = T\backslash\{\alpha\}|E^i) = \binom{n-1}{n^{1/2}-1}^{-1}$. Hence we have

$$\begin{aligned}
&\Pr(S_{k_0} = T)\\
=&\Pr(E_1) \cdot \sum_{i\in[w]} \Pr(E^i) \cdot \tbinom{n-1}{n^{1/2}-1}^{-1} + \Pr((S_{j_0} = T) \cap \bar{E})\\
=&\Pr(E) \cdot \tbinom{n-1}{n^{1/2}-1}^{-1} + \Pr((S_{j_0} = T) \cap \bar{E}).
\end{aligned}$$

Now we consider the distribution of S_{k_1}. For any $T \subseteq [\![n]\!]$ subject to $|T| = n^{1/2}$, we have

$$\begin{aligned}
&\Pr(S_{k_1} = T)\\
=&\Pr((S_{w+u} = T) \cap E) + \Pr((S_{j_0} = T) \cap \bar{E}).\\
=&\Pr(E) \cdot \Pr((S_{w+u} = T)|E) + \Pr((S_{j_0} = T) \cap \bar{E}).
\end{aligned}$$

When the protocol succeeds, we know that $S_{w+u} = S \cup \{\alpha\}$, with S being a random size-$(n^{1/2} - 1)$ subset of $[\![n]\!]\backslash\{\alpha\}$. If $\alpha \notin T$, then $\Pr((S_{w+u} = T)|E) = 0$, which implies that

$$\Pr(S_{k_1} = T) = \Pr((S_{j_0} = T) \cap \bar{E}).$$

This means that $\Pr(S_{k_0} = T) = \Pr(S_{k_1} = T)$ for the case that $\alpha \notin T$. If $\alpha \in T$, then

$$\Pr(S_{k_1} = T|E) = \Pr(S = T\backslash\{\alpha\}|E) = \tbinom{n-1}{n^{1/2}-1}^{-1}.$$

Therefore, we have

$$\Pr(S_{k_1} = T) = \Pr(E) \cdot \tbinom{n-1}{n^{1/2}-1}^{-1} + \Pr((S_{j_0} = T) \cap \bar{E}).$$

Namely, $\Pr(S_{k_0} = T) = \Pr(S_{k_1} = T)$ if $\alpha \in T$. Based on our discussion, $\Pr(S_{k_0} = T) = \Pr(S_{k_1} = T)$ for any $T \subseteq [\![n]\!]$ subject to $|T| = n^{1/2}$. Therefore, S_{k_0} and S_{k_1} have the same distribution. □

C.1 Correctness Proof

We prove that for each query, the protocol is correct with probability at least 1/2.

Proof. We first show that the u-th query will succeed if $b = 1$ and $\alpha \in S_k$ for some $k \in M_u$. Without loss of generality, let (S_k, h_k) be the consumed hint, where $h_k = r_k \oplus \sum_{i\in S_k} \mathsf{DB}_i$. We need to show that $a_0 \oplus a_1 \oplus h_k = \mathsf{DB}_\alpha$. Note that

$$\begin{aligned}
&a_0 \oplus a_1 \oplus h_k\\
=&(\langle \boldsymbol{v}_0, \bar{\boldsymbol{r}}\rangle \oplus \tilde{r}) \oplus (\langle \boldsymbol{v}_1, \bar{\boldsymbol{r}}\rangle \oplus h^* \oplus \tilde{r}) \oplus h_k\\
=&\langle \boldsymbol{v}_0 \oplus \boldsymbol{v}_1, \bar{\boldsymbol{r}}\rangle \oplus h^* \oplus h_k\\
=&\langle \boldsymbol{e}_{k+1}^{w+u}, \bar{\boldsymbol{r}}\rangle \oplus h^* \oplus h_k.
\end{aligned}$$

Since $\bar{\boldsymbol{r}} = (\bar{r}, \boldsymbol{r}) = (\bar{r}, r_1, \ldots, r_{w+u-1})$ when handling the u-th query, we have $\langle \boldsymbol{e}_{k+1}^{w+u}, \bar{\boldsymbol{r}} \rangle = r_k$, implying that

$$a_0 \oplus a_1 \oplus h_k = r_k \oplus \sum_{i \in S^*} \mathsf{DB}_i \oplus r_k \oplus \sum_{i \in S_k} \mathsf{DB}_i = \sum_{i \in S^*} \mathsf{DB}_i \oplus \sum_{i \in S_k} \mathsf{DB}_i.$$

Note that $S^* = S_k \backslash \{\alpha\}$ when $b = 1$, we have $a_0 \oplus a_1 \oplus h_k = \mathsf{DB}_\alpha$.

It remains to show that for any $u \in [\beta]$ the probability that $b = 1$ and $\alpha \in S_k$ for some $k \in M_u$ is at least $1/2$. We begin by proving this for the *first* query in the following claim.

Claim. For any $\alpha \in [\![n]\!]$, the probability that $b = 0$ or $\alpha \notin S_j$ for all $j \in [w]$ is at most $1/2$.

Proof. Denote E_1 the event that $b = 0$ and E_2 the event that $\alpha \notin S_j$ for all $j \in [w]$. When E_1 or E_2 happens, the query will fail. We need to show that $\Pr(E_1 \cup E_2) \leq 1/2$. First, note that $\Pr(E_1) = (n^{1/2} - 1)/n$. Since $(n^{1/2} - 1)/n$ reaches its maximum at $n = 4$, we have $\Pr(E_1) \leq 1/4$. Moreover, since S_j is a size-$n^{1/2}$ random subset of $[\![n]\!]$, we know that $\Pr(E_2) = (1 - n^{1/2}/n)^w = (1 - n^{-1/2})^w \leq e^{-w/n^{1/2}}$. Note that $w = (\ln 4) n^{1/2}$, hence $\Pr(E_2) \leq e^{-\ln 4} = 1/4$. Therefore, we have $\Pr(E_1 \cup E_2) \leq \Pr(E_1) + \Pr(E_2) \leq 1/2$. □

By Lemma 1, the hints in every M_u have the same distribution. Combining the above claim, we can conclude that for any $\alpha \in [\![n]\!]$ and $u \in [\beta]$, the probability that $b = 0$ or $\alpha \notin S_j$ for all $j \in M_u$ is at most $1/2$, which implies that each query will succeed with probability at least $1/2$. □

C.2 Client Privacy Proof

We now show that in each copy, the client obtains only the retrieved values if the query succeeds and nothing otherwise.

Proof. We show that there exists a simulator Sim taking the input and output of the client can generate a simulated view that is statistically indistinguishable from the real view of the client. Concretely, our simulator Sim works as follows.

Simulated offline stage.

1. Sample w random values $h_1, \ldots, h_w \in \mathcal{D}$ and simulate the left server sending $\{h_j\}_{j \in [w]}$ to the client.
2. Store $\{h_j\}_{j \in [w]}$ and initialize $M = [w]$.

Simulated online stage. For the u-th query α,

- If the output of the client is $x_\alpha \neq \perp$,
  1. Sample a random value $h_{w+u} \in \mathcal{D}$.
  2. Choose some $k \in M$ and sample two random values $a_0, a_1 \in \mathcal{D}$ subject to $a_0 \oplus a_1 = x_\alpha \oplus h_k$.
  3. Simulate the left server sending (h_{w+u}, a_0) to the client; simulate the right server sending a_1 to the client.
  4. Update $M \leftarrow M \cup \{w + u\} \backslash \{k\}$.
- If the output of the client is $\perp$,
  1. Sample three random values $h, a_0, a_1 \in \mathcal{D}$;
  2. Simulate the left server sending (h, a_0) to the client; simulate the right server sending a_1 to the client.

Now, we show that the real and simulated views are statistically indistinguishable. In the real view, the client receives w hint values $\{h_j\}_{j \in [w]}$ from the left server during the offline stage with each $h_j = r_j \oplus \sum_{i \in S_j} \mathsf{DB}_i$. Since each $r_j \in \mathcal{D}$ is a random value that is unknown to the client, h_j is also a random value. Hence the simulation for the offline stage is perfect. In the online stage, the client receives h, a_0 from the left server and a_1 from the right server, where $h = r \oplus \sum_{i \in S^h} \mathsf{DB}_i, a_0 = \langle \boldsymbol{v}_0, \bar{\boldsymbol{r}} \rangle \oplus \tilde{r}, a_1 = \langle \boldsymbol{v}_1, \bar{\boldsymbol{r}} \rangle \oplus h^* \oplus \tilde{r}$. Since r is random, h is also random, implying that our simulation for h is perfect. As for the simulation for a_0, a_1, we consider two cases: the query succeeds or fails.

1. *When the query succeeds*, we have $a_0 \oplus a_1 \oplus h_k = \mathsf{DB}_\alpha$ for some $k \in M$. Since $\tilde{r}$ is a random value, hence a_0 and a_1 are in fact two random values conditioned on $a_0 \oplus a_1 = \mathsf{DB}_\alpha \oplus h_k$. In particular, we update M to ensure that the h_k we used in the simulation has not been used for simulating a_0, a_1 in previous online queries. Therefore, our simulation for a_0, a_1 is perfect.

1. *When the query fails*, notice that

$$\begin{aligned}
&a_0 \oplus a_1 \\
=&(\langle \boldsymbol{v}_0, \bar{\boldsymbol{r}} \rangle \oplus \tilde{r}) \oplus (\langle \boldsymbol{v}_1, \bar{\boldsymbol{r}} \rangle \oplus h^* \oplus \tilde{r}) \\
=&\langle \boldsymbol{v}_0 \oplus \boldsymbol{v}_1, \bar{\boldsymbol{r}} \rangle \oplus h^* \\
=&\langle \boldsymbol{e}_{k+1}^{w+u}, \bar{\boldsymbol{r}} \rangle \oplus h^*.
\end{aligned}$$

Here $\bar{\boldsymbol{r}} = (\bar{r}, \boldsymbol{r}) = (\bar{r}, r_1, \ldots, r_{w+u-1})$. If the query fails, we have $k = 0$, which implies that $\langle \boldsymbol{e}_{k+1}^{w+u}, \bar{\boldsymbol{r}} \rangle = \langle \boldsymbol{e}_1^{w+u}, \bar{\boldsymbol{r}} \rangle = \bar{r}$. Since $\bar{r}$ is a random value, we know that $a_0 \oplus a_1 = \bar{r} \oplus h^*$ is also a random value. That is, a_0, a_1 are in fact two random values, hence our simulation for a_0, a_1 is also perfect. This completes the client privacy proof. □

C.3 Left-Server Privacy Proof

Proof. In the offline stage, the left server receives w random size-$n^{1/2}$ subsets of $[\![n]\!]$. This can be simulated perfectly. In the online stage, the left server receives the query message $(S^h, \boldsymbol{v}_0)$. Note that $\boldsymbol{v}_0$ is a random binary vector and thus can be simulated perfectly. As for the set S^h, we next show that it is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!]$.

By the protocol, we know that S^h is obtained by removing α^* from $S_{\mathsf{new}} = S \cup \{\alpha\}$, where S is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!] \backslash \{\alpha\}$. If $b = 1$, we have $\alpha^* = \alpha$, and if $b = 0$, then α^* is a random element in S. For any $T \subseteq [\![n]\!]$ subject to $|T| = n^{1/2} - 1$, if $\alpha \in T$, then we have

$$\begin{aligned}
&\Pr(S^h = T) \\
=&\Pr(S_{\mathsf{new}} \backslash \{\alpha^*\} = T) \\
=&\Pr(b = 1) \cdot \Pr(S = T | b = 1) + \Pr(b = 0) \cdot \Pr(S_{\mathsf{new}} \backslash \{\alpha^*\} = T | b = 0) \\
=&0 + \frac{n^{1/2} - 1}{n} \cdot \Pr(S \backslash \{\alpha^*\} = T \backslash \{\alpha\} | b = 0).
\end{aligned}$$

Note that if $b = 0$, then α^* is a random index in S. Hence $S \backslash \{\alpha^*\}$ is a random size-$(n^{1/2}-2)$ subset of $[\![n]\!] \backslash \{\alpha\}$, which implies that

$$\Pr(S \backslash \{\alpha^*\} = T \backslash \{\alpha\} | b = 0) = \binom{n-1}{n^{1/2}-2}^{-1}.$$

Therefore, we have

$$\Pr(S^h = T) = \frac{n^{1/2} - 1}{n} \cdot \binom{n-1}{n^{1/2}-2}^{-1} = \binom{n}{n^{1/2}-1}^{-1}.$$

Now we consider the case that $\alpha \notin T$. We have

$$\begin{aligned}
&\Pr(S^h = T) \\
=&\Pr(b = 1) \cdot \Pr(S = T | b = 1) + \Pr(b = 0) \cdot \Pr(S_{\mathsf{new}} \backslash \{\alpha^*\} = T | b = 0) \\
=&(1 - \frac{n^{1/2} - 1}{n}) \cdot \Pr(S = T | b = 1) + 0 \\
=&(1 - \frac{n^{1/2} - 1}{n}) \cdot \binom{n-1}{n^{1/2}-1}^{-1} \\
=&\binom{n}{n^{1/2}-1}^{-1}.
\end{aligned}$$

Based on the above discussion, we can conclude that S^h is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!]$. Thus the simulator can simulate S^h perfectly. □

C.4 Right-Server Privacy Proof

Proof. The right server receives nothing from the client during the offline stage. In the online stage, the right server receives the query message $(S^*, \boldsymbol{v}_1)$. Note that $\boldsymbol{v}_1$ is a random binary vector and can

be simulated perfectly. As for the set S^*, we will show that it is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!]$.

By the protocol, we know that S^* is obtained by removing α^* from S_{query}. When the query fails (i.e., $b=0$ or $k=0$), we have $S_{\mathsf{query}} = S_{\mathsf{new}} = S \cup \{\alpha\}$ with S being a random size-$(n^{1/2}-1)$ subset of $[\![n]\!]\backslash\{\alpha\}$. When the query succeeds, we have $S_{\mathsf{query}} = S_k$ with $k \in M$ and $\alpha \in S_k$. Note that $k \in M$ means that S_k has not been used in previous queries, hence from the view of the right server (which knows nothing about the hint table), S_k is a fresh random size-$n^{1/2}$ subset of $[\![n]\!]$ subject to $\alpha \in S_k$. That is, whether the query succeeds or not, S_{query} is a random size-$n^{1/2}$ subset of $[\![n]\!]$ subject to $\alpha \in S_{\mathsf{query}}$. As a result, using a similar analysis in our left-server privacy proof, we can prove that S^* is a random size-$(n^{1/2}-1)$ subset of $[\![n]\!]$. Thus the simulator can simulate S^* perfectly. □

D Complexity Analysis

In this section, we provide detailed complexity analyses for our protocols. We will analyze the complexity of shared randomness separately, without factoring it into the cost of the offline stage.

D.1 Complexity of the First Single-Query Protocol

Let $l = \log|\mathbb{F}|$ with $\mathbb{F}$ being the field that the entries lie in.

1. Shared randomness. The servers need to share and store $w+2 = O(n^{1/2})$ random field elements, hence the extra server-storage is $O(ln^{1/2})$.

2. Offline complexity. In the offline stage, the client samples S and $\delta_1, \ldots, \delta_w$ and sends them to the left server, which takes $O(n^{1/2}\log n + w\log n) = \tilde{O}(n^{1/2})$ computation and communication. Then the left server computes and sends the masked hints $\{h_j\}_{j\in[w]}$ to the client, which takes $wln^{1/2} = O(ln)$ computation and $wl = O(ln^{1/2})$ communication. As a result, the total computation and communication are $\tilde{O}(ln)$ and $\tilde{O}(ln^{1/2})$, respectively. Finally, storing $(S, \{(\delta_j, h_j)\}_{j\in[w]})$ requires $n^{1/2}\log n + w(\log n + l) = \tilde{O}(ln^{1/2})$ client-storage.

3. Online complexity. In the online stage, the client first samples a bit b that equals 0 with probability $(n^{1/2}-1)/n$, which takes $O(\log n)$ computation. Then the client checks whether $\alpha - \delta_j \notin S$ for all $j \in [w]$. As we noted before, this can be done using only $\tilde{O}(n^{1/2})$ computation. After the check procedure, the client needs to compute S_{query}, which takes $O(n^{1/2}\log n) = \tilde{O}(n^{1/2})$ computation. Next, the client needs to sample α^*, which takes $O(\log n)$ computation. Furthermore, the client samples $\alpha_0 \in \mathbb{F}$ and computes α_1, which takes $O(l)$ computation. In total, the computation of generating the query messages is $\tilde{O}(n^{1/2}+l)$. Finally, it is easy to see that sending $\alpha_0, S^*, k, \alpha_1$ takes $\tilde{O}(n^{1/2}+l)$ communication.

Upon receiving α_0, the left server computes and sends $a_0 = \hat{r}\cdot\alpha_0 + \tilde{r}$ to the client. This takes $O(l)$ computation and communication. Upon receiving (S^*, k, α_1) (if $k \neq 0$), the right server computes $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ and $a_1 = \hat{r}\cdot\alpha_1 - r_k - h^* - \tilde{r}$ and sends a_1 to the client. This takes $O(ln^{1/2})$ computation and $O(l)$ communication. Moreover, the Extract procedure takes $O(l)$ computation.

D.2 Complexity of the Second Single-Query Protocol

Let l be the entry length.

1. Shared randomness. The servers need to share and store $w+2 = O(n^{1/2})$ random values, hence the extra server-storage is $O(ln^{1/2})$.

2. Offline complexity. In the offline stage, the client samples S and $\delta_1, \ldots, \delta_w$ and sends them to the left server, which takes $O(n^{1/2}\log n + w\log n) = \tilde{O}(n^{1/2})$ computation and communication. Then the left server computes and sends the masked hints $\{h_j\}_{j\in[w]}$ to the client, which takes $wln^{1/2} = O(ln)$ computation and $wl = O(ln^{1/2})$ communication. As a result, the total computation and communication are $\tilde{O}(ln)$ and $\tilde{O}(ln^{1/2})$, respectively. Finally, storing the hints requires $n^{1/2}\log n + w(\log n + l) = \tilde{O}(ln^{1/2})$ client-storage.

3. Online complexity. In the online stage, the client first samples a bit b that equals 0 with probability $(n^{1/2}-1)/n$, which takes $O(\log n)$ computation. Then the client checks whether $\alpha - \delta_j \notin S$ for all $j \in$

$[w]$. This can be done using only $\tilde{O}(n^{1/2})$ computation. After the check procedure, the client needs to compute S_{query}, which takes $O(n^{1/2}\log n) = \tilde{O}(n^{1/2})$ computation. Next, the client needs to sample α^*, which takes $O(\log n)$ computation. Furthermore, the client needs to sample $\boldsymbol{v}_0 \in \mathbb{F}_2^{w+1}$ and computes $\boldsymbol{v}_1$, which takes $O(w) = O(n^{1/2})$ computation. In total, the computation of generating the query messages is $\tilde{O}(n^{1/2})$. Finally, it is easy to see that sending $\boldsymbol{v}_0, S^*, \boldsymbol{v}_1$ takes $\tilde{O}(n^{1/2})$ communication.

Upon receiving $\boldsymbol{v}_0$, the left server computes and sends $a_0 = \langle \boldsymbol{v}_0, \boldsymbol{r}\rangle \oplus \tilde{r}$ to the client. This takes $O(lw) = O(ln^{1/2})$ computation and $O(l)$ communication. Upon receiving $(S^*, \boldsymbol{v}_1)$, the right server computes $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ and $a_1 = \langle \boldsymbol{v}_1, \boldsymbol{r}\rangle \oplus h^* \oplus \tilde{r}$ and sends a_1 to the client. This takes $O(ln^{1/2} + lw) = O(ln^{1/2})$ computation and $O(l)$ communication. Moreover, the $\mathsf{Extract}$ procedure takes $O(l)$ computation.

D.3 Complexity of the Multi-Query Protocol with Linear Client-Storage

Let l be the entry length.

1. Shared randomness. The servers need to share and store $w + 3\beta = O(n^{1/2} + \beta)$ random values, hence the extra server-storage is $O(ln^{1/2} + l\beta)$.

2. Offline complexity. In the offline stage, the client samples $S_1, \ldots, S_w$ and sends them to the left server, which takes $O(wn^{1/2}\log n) = \tilde{O}(n)$ computation and communication. Then the left server computes and sends the masked hints $\{h_j\}_{j\in[w]}$ to the client, which takes $wln^{1/2} = O(ln)$ computation and $wl = O(ln^{1/2})$ communication. As a result, the total computation and communication are $\tilde{O}(ln)$ and $\tilde{O}(n + ln^{1/2})$, respectively. Finally, storing the hints requires $w(n^{1/2}\log n + l) = \tilde{O}(n + ln^{1/2})$ client-storage.

3. Online complexity. We analyze the complexity of handling the u-th query. At step (a), the client first samples a bit b that equals 0 with probability $(n^{1/2} - 1)/n$, which takes $O(\log n)$ computation. Then the client samples S and computes S_{new}, which takes $O(n^{1/2}\log n) = \tilde{O}(n^{1/2})$ computation. At step (c), the client checks whether $\alpha \notin S_j$ for all $j \in M$. If we sort every S_j during the offline stage (which takes $w\tilde{O}(n^{1/2}) = \tilde{O}(n)$ computation using Quicksort), then checking whether $\alpha \notin S_j$ takes only $O(\log n)$ computation using binary search. Since $|M| = w$, we know that the computation of step (c) is $O(w\log n) = \tilde{O}(n^{1/2})$. At step (d), the client needs to sample α^*, which takes $O(\log n)$ computation. Moreover, at step (e), the client samples $\boldsymbol{v}_0 \in \mathbb{F}_2^{w+u}$ and computes $\boldsymbol{v}_1 = \boldsymbol{e}_{k+1}^{w+u} \oplus \boldsymbol{v}_0$. This takes $O(w+u) = O(n^{1/2}+u)$ computation. Finally, sending $(S^h, \boldsymbol{v}_0), (S^*, \boldsymbol{v}_1)$ takes $2n^{1/2}\log n + 2(w+u) = \tilde{O}(n^{1/2} + u)$ communication.

Upon receiving $(S^h, \boldsymbol{v}_0)$, the left server needs to compute and send $h = r \oplus \sum_{i\in S^h} \mathsf{DB}_i, a_0 = \langle \boldsymbol{v}_0, \bar{\boldsymbol{r}}\rangle \oplus \tilde{r}$ to the client. This takes $O(ln^{1/2} + l(w+u)) = O(ln^{1/2} + lu)$ computation and $O(l)$ communication. Upon receiving $(S^*, \boldsymbol{v}_1)$, the right server computes $h^* = \sum_{i\in S^*} \mathsf{DB}_i$ and $a_1 = \langle \boldsymbol{v}_1, \bar{\boldsymbol{r}}\rangle \oplus h^* \oplus \tilde{r}$ and sends a_1 to the client. This takes $O(ln^{1/2} + l(w+u)) = O(ln^{1/2} + lu)$ computation and $O(l)$ communication. Moreover, the $\mathsf{Extract}$ and $\mathsf{Refresh}$ procedures take $O(l)$ computation.

D.4 Complexity of the Multi-Query Protocol with Sublinear Client-Storage

Let l be the entry length.

1. Shared randomness. The servers need to share and store $n_r w + 3\beta n_r + 2n_r + 2 = O(n_r(n_c^{1/2} + \beta))$ random values, hence the extra server-storage is $O(ln_r(n_c^{1/2} + \beta))$.

2. Offline complexity. In the offline stage, the client samples $S_1, \ldots, S_w$ and sends them to the left server, which takes $O(wn_c^{1/2}\log n_c) = O(n_c \log n_c)$ computation and communication. Then the left server computes and sends the masked hints $\{h_{t,j}\}_{t\in[n_r], j\in[w]}$ to the client, which takes $n_r w l n_c^{1/2} = O(ln_r n_c)$ computation and $n_r wl = O(ln_r n_c^{1/2})$ communication. As a result, the total computation and communication are $O(ln_r n_c + n_c \log n_c)$ and $O(n_c \log n_c + ln_r n_c^{1/2})$, respectively. Finally, storing the hints requires $w(n_c^{1/2}\log n_c + ln_r) = O(n_c \log n_c + ln_r n_c^{1/2})$ client-storage.

3. Online complexity. We analyze the complexity of handling the u-th query. At step (a) during the Query procedure, the client samples a bit b that equals 0 with probability $(n_c^{1/2} - 1)/n_c$, which takes $O(\log n_c)$

computation. Then the client samples S and computes S_{new}, which takes $O(n_c^{1/2} \log n_c)$ computation. At step (c), the client checks whether $\alpha_c \notin S_j$ for all $j \in M$. This takes $O(w \log n_c) = O(n_c^{1/2} \log n_c)$ computation using binary search if the client has sorted every S_j during the offline stage. At step (d), the client needs to sample α_c^*, which takes $O(\log n_c)$ computation. Moreover, at step (e), the client samples $\boldsymbol{v}_0 \in \mathbb{F}_2^{w+u}$ and computes $\boldsymbol{v}_1 = \boldsymbol{e}_{k+1}^{w+u} \oplus \boldsymbol{v}_0$. This takes $O(w+u) = O(n_c^{1/2} + u)$ computation. And at step (f), the client samples $\boldsymbol{b}_0 \in \mathbb{F}_2^{n_r}$ and computes $\boldsymbol{b}_1 = \boldsymbol{e}_{\alpha_r}^{n_r} \oplus \boldsymbol{v}_0$, which takes $O(n_r)$ computation. Finally, sending $(S^h, \boldsymbol{v}_0, \boldsymbol{b}_0), (S^*, \boldsymbol{v}_1, \boldsymbol{b}_1)$ takes $2n_c^{1/2} \log n_c + 2(w+u) + 2n_r = O(n_c^{1/2} \log n_c + n_r + u)$ communication.

Upon receiving $(S^h, \boldsymbol{v}_0, \boldsymbol{b}_0)$, the left server needs to compute $h_t = r_t \oplus \sum_{i \in S^h} \mathsf{RB}_{t,i}, a_{0,t} = \langle \boldsymbol{v}_0, \bar{\boldsymbol{r}}_t \rangle \oplus \tilde{r}_t$ for all $t \in [n_r]$. This takes $O(n_r(ln_c^{1/2} + l(w+u))) = O(ln_r(n_c^{1/2} + u))$ computation. Then the left server computes $\boldsymbol{h} = (h_1, \ldots, h_{n_r}), \bar{\boldsymbol{a}}_0 = \boldsymbol{a}_0 \oplus \boldsymbol{\eta}_0, c_0 = \langle \boldsymbol{\eta}_0, \boldsymbol{b}_0 \rangle \oplus \bar{\eta}, d_0 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_0 \rangle \oplus \tilde{\eta}$, where $\boldsymbol{a}_0 = (a_{0,1}, \ldots, a_{0,n_r})$. This takes $O(ln_r)$ computation. The left server needs to send $(\boldsymbol{h}, \bar{\boldsymbol{a}}_0, c_0, d_0)$, which takes $O(ln_r)$ communication.

For the right server, upon receiving $(S^*, \boldsymbol{v}_1, \boldsymbol{b}_1)$, it needs to compute $h_t^* = \sum_{i \in S^*} \mathsf{RB}_{t,i}$ and $a_{1,t} = \langle \boldsymbol{v}_1, \bar{\boldsymbol{r}}_t \rangle \oplus h_t^* \oplus \tilde{r}_t$ for all $t \in [n_r]$. This takes $O(n_r(ln_c^{1/2} + l(w+u))) = O(ln_r(n_c^{1/2} + u))$ computation. Then the right server computes $\bar{\boldsymbol{a}}_1 = \boldsymbol{a}_1 \oplus \boldsymbol{\eta}_1, c_1 = \langle \boldsymbol{\eta}_0, \boldsymbol{b}_1 \rangle \oplus \bar{\eta}, d_1 = \langle \boldsymbol{\eta}_1, \boldsymbol{b}_1 \rangle \oplus \tilde{\eta}$, where $\boldsymbol{a}_1 = (a_{1,1}, \ldots, a_{1,n_r})$. This takes $O(ln_r)$ computation. The right server needs to send $(\bar{\boldsymbol{a}}_1, c_1, d_1)$, which takes $O(ln_r)$ communication.

The $\mathsf{Extract}$ procedure requires the client to compute $a_0 = \langle \bar{\boldsymbol{a}}_0, \boldsymbol{e}_{\alpha_r} \rangle \oplus c_0 \oplus c_1, a_1 = \langle \bar{\boldsymbol{a}}_1, \boldsymbol{e}_{\alpha_r} \rangle \oplus d_0 \oplus d_1$ and $x_\alpha = a_0 \oplus a_1 \oplus h_{\alpha_r, k}$. Note that $\boldsymbol{e}_{\alpha_r}$ is a unit vector with only one non-zero entry, hence the required computation is $O(l)$. Finally, the $\mathsf{Refresh}$ procedure takes $O(ln_r)$ computation, as the client needs to update n_r hints values.